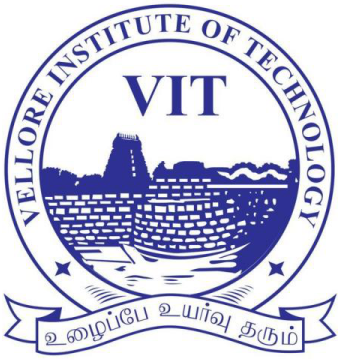
VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



BCSE307P – COMPILER DESIGN LAB

PROJECT REPORT — REVIEW 3

CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset

Submitted by

Srikiran – 24BCE2215

Dhruv – 24BCE2211

Prince – 24BDS0153

Under the guidance of

Nithya K

Assistant Professor Senior Grade II

OCTOBER, 2026

BONAFIDE CERTIFICATE

This is to certify that the project work titled **CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset** is a bonafide record of work carried out by **Srikiran (24BCE2215), Dhruv (24BCE2211), Prince (24BDS0153)**, submitted in partial fulfilment of the requirements for the course BCSE307P – Compiler Design Lab, School of Computer Science and Engineering, VIT Vellore, during the academic year 2026 - 2027.

Faculty Guide: _____

Date: _____

DECLARATION

We hereby declare that the project report titled **CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset** submitted by us to VIT Vellore in partial fulfilment of the requirement for the award of the degree/course BCSE307P – Compiler Design Lab is a record of bona fide work carried out by us under the supervision of Nithya K. The contents of this report, in full or in part, have not been submitted to any other institute or university for the award of any degree or diploma.

Place: _____

Date: _____

Signatures of all team members

ABSTRACT

CodeTrace is a static program-analysis platform for CT-C, a documented subset of the C language, built to make the reasoning behind a security result visible. A hand-written lexer produces tokens with source positions, and a parser generator built for the project constructs canonical LR(1), LALR(1) or SLR(1) tables for the same grammar. The resulting syntax tree is resolved by a scoped semantic analyser that assigns stable identifiers to declarations, and those identifiers drive per-function control-flow graphs and a shared worklist framework that computes reaching definitions, liveness and definite assignment. A declarative rule language, parsed by the same LR machinery, declares taint sources, sinks, sanitizers and validators; the analysis follows explicit data flow through assignments, arguments and return values, and relates each result to a typed dependency graph and to program slices. A nine-view browser workbench and an HTTP API present the compiler artefacts and the security conclusions together, and a run can be exported as stable JSON or as a self-contained HTML document. This report records the Review 3 state of the work: 350 engine tests, 53 interface tests, 69 independent audit checks and 18 of 18 injected defects are detected, and the labelled corpus of 33 programs is reproduced at precision 1.0 and recall 1.0. The contribution is inspectability within a bounded language: every reported flow can be traced back to the tokens, statements and analysis facts that produced it.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 10
1.1 Background 10
1.2 Problem Statement 11
1.3 Objectives 11
1.4 Scope of the Work 12
1.5 Organisation of the Report 12
CHAPTER 2 LITERATURE SURVEY 13
2.1 Compiler Construction 13
2.2 Data-Flow Analysis 14
2.3 Program Dependence and Slicing 14
2.4 Static Analysis for Security 15
2.5 Research Gap 15
2.6 Positioning of the Present Work 16
CHAPTER 3 SYSTEM DESIGN 17
3.1 Architecture Overview 17
3.2 Analysis Pipeline 18
3.3 Module Description 19
3.4 Compiler Phases 20
3.5 Rule Language Design 23
3.6 Data Model and Interface Contract 25
3.7 Interface Design 26

3.8 Design Decisions and Trade-offs 26
CHAPTER 4 IMPLEMENTATION DETAILS 28
4.1 Technology Stack 28
4.2 Repository Organisation 28
4.3 Implementation Roadmap and Review 3 Deliverables 30
4.4 Work File Explanation 30
4.5 Module-Wise Reference 40
4.6 Worked Example: Command Injection 43
4.7 Diagnostics and Partial Analysis 46
4.8 Reproducing the Results 47
CHAPTER 5 RESULTS AND DISCUSSION 49
5.1 Application Walkthrough 49
5.2 Features Implemented 64
5.3 Verification and Testing 65
5.4 Evaluation Results 66
5.5 Discussion 67
5.6 Limitations and Threats to Validity 68
5.7 Current Progress and Challenges 69
5.8 Conclusion and Future Enhancements 69
References 70

LIST OF FIGURES

Figure 3.1: Runtime architecture: the browser workbench, the HTTP API and the analysis engine. ... 17
Figure 3.2: The analysis pipeline: each stage consumes the artefacts produced by the stage above it. ... 18
Figure 4.1: The reported flow from source to sink, drawn from the run whose evidence path is tabulated above. ... 44
Figure 4.2: The typed dependency graph of the worked example, drawn from the graph the engine computed: ten nodes, ten edges, and the reported source-to-sink path in dashes. ... 45
Figure 5.1: Overview of the Workbench. 49
Figure 5.2: Analysis Workbench. 50
Figure 5.3: Editor State After a Run. 51
Figure 5.4: Located Syntax Diagnostics. 51
Figure 5.5: A Refused Analysis. 52
Figure 5.6: Findings with Evidence. 53
Figure 5.7: The Ordered Evidence Path. 54
Figure 5.8: Program Dependency Graph. 56
Figure 5.9: Data-Flow Analysis. 56
Figure 5.10: Program Slicing. 57
Figure 5.11: The Induced Subgraph. 58
Figure 5.12: Rule Editor and Validation. 59
Figure 5.13: Validation Result and Diagnostics. 60
Figure 5.14: Report Export and Run Comparison. 61
Figure 5.15: Documentation View. 62
Figure 5.16: Dark Theme. 63

LIST OF TABLES

Table 2.1: Summary of the literature and its use in the implementation. 16
Table 3.1: Pipeline stages, their inputs and their outputs. 19
Table 3.2: Package responsibilities and principal outputs. 20
Table 3.3: Table construction for the same grammar in the three supported modes. ... 21
Table 3.4: The built-in rule set. 24
Table 3.5: Principal HTTP endpoints. 25
Table 3.6: The nine views and their purpose. 26
Table 3.7: Principal design decisions and their consequences. 27
Table 4.1: Technology choices and the reasons behind them. 28
Table 4.2: Repository layout, with file and line counts measured from this checkout. ... 29
Table 4.3: Deliverables by review cycle. 30
Table 4.4: Compiler front end (codetrace/frontend): 9 files, 3,277 lines. 31
Table 4.5: Semantic analysis (codetrace/semantics): 4 files, 1,352 lines. 31
Table 4.6: Rule language (codetrace/rules): 4 files, 1,235 lines. 31
Table 4.7: Program analyses (codetrace/analysis): 11 files, 4,130 lines. 32
Table 4.8: Findings and reporting (codetrace/findings, codetrace/reporting): 6 files, 1,875 lines. ... 32
Table 4.9: HTTP API (codetrace/api): 4 files, 1,152 lines. 33

Table 4.10: Package root (codetrace/): 2 files, 22 lines. 33
Table 4.11: Interface (frontend/): 21 files, 4,350 lines. 34
Table 4.12: Interface tests (frontend/tests): 5 files, 2,115 lines. 35
Table 4.13: Engine tests (tests/): 18 files, 6,212 lines. 36
Table 4.14: Corpus and harness (evaluation/): 2 files, 754 lines. 36
Table 4.15: Tooling (scripts/): 16 files, 9,300 lines. 37
Table 4.16: Example programs (examples/): 34 files, 307 lines. 38
Table 4.17: Documentation (docs/): 13 files, 4,050 lines. 40
Table 4.18: Front-end and semantic modules. 41
Table 4.19: Analysis modules. 42
Table 4.20: Rule, finding, reporting and interface modules. 43
Table 4.21: The evidence path reported for the worked example. 44
Table 4.22: Representative diagnostics, their phases and their effect on the run. ... 47
Table 5.1: Features implemented, with the module that provides each and the evidence for it. ... 65
Table 5.2: Verification instruments and their results on this checkout. 66
Table 5.3: Composition of the labelled corpus and the outcome for each group. ... 67
Table 5.4: Limitations, their effect on a result and where they are disclosed. 68

CHAPTER 1
INTRODUCTION

1.1 Background

A compiler front end translates source text into a structured representation that later stages can examine. Lexical analysis groups characters into tokens, syntax analysis decides whether the token sequence belongs to the language defined by a grammar, and semantic analysis resolves declarations and verifies the relationships between names, types and operations. These stages are conventionally viewed as the preparation for code generation, but they are equally the preparation for program analysis: once a program exists as an abstract syntax tree with resolved identifiers, statements can be compared, values can be followed and the effect of a branch can be described [1].

Static analysis derives properties of a program without executing it. Data-flow analysis computes facts at each program point by propagating information along the control-flow edges until a fixed point is reached, which makes it possible to state, for example, which assignments may reach a read or which variables still hold a value that will be used [4], [5]. Taint analysis applies the same machinery to a security question: it records where untrusted values enter a program, how they are transformed, and whether they arrive at an operation that requires particular handling, such as a command interpreter, a database query, a file path or a format-string function [9], [13].

The two traditions meet in the tool described in this report. CodeTrace analyses CT-C, a small C-like language whose grammar, types, statements and diagnostics are documented in the repository. The lexer, the parser generator, the symbol tables, the control-flow graphs, the data-flow solvers and the dependency graph are all part of the project rather than borrowed from a general-purpose compiler, and each of them is exposed to the reader through a browser workbench and an HTTP API. The intention is educational as much as technical: a finding should be accompanied by the reasoning that produced it, and the reasoning should be checkable against the artefacts of the compiler pipeline.

1.2 Problem Statement

A security report that names a rule and a line number asks the reader to accept a conclusion without a demonstration. The reader cannot tell whether the tool understood the syntax of the program, whether it resolved the identifiers correctly, whether the value could actually travel from the source to the sink, or whether an analysis approximation decided the matter. A second difficulty is the empty result. A tool that reports nothing is silent about its own coverage, so the absence of a finding is easily read as evidence that the program is safe, which is a stronger claim than static analysis can support.

The project therefore treats the following as its problem. Given one source file written in the supported subset, construct the compiler representations that analysis requires, derive the control-flow and data-flow facts that the analysis depends on, identify the potential source-to-sink flows described by a declarative rule set, and present each conclusion together with the path, the certainty and the limitations that qualify it. The analysed source must remain data throughout; it is parsed and inspected, never executed. The Review 3 requirement adds a further constraint: the artefacts must be reachable from the interface without reading the source code, and the claims the project makes about itself must be reproducible by a reader who has only the repository.

1.3 Objectives

The work set out to achieve the following objectives.

1. Implement a lexer that preserves byte offsets and line and column positions, and that reports malformed or unsupported input with a located diagnostic.
2. Generate LR(1)-family parsing tables from a grammar written for the project, and build a syntax tree in which every node carries its source span.
3. Resolve declarations with nested symbol tables and assign each declaration a stable identifier, so that later stages refer to a symbol rather than to a name.
4. Construct per-function control-flow graphs and compute reaching definitions, liveness and definite assignment with a single reusable solver.
5. Express taint sources, sinks, sanitizers, validators and flows in a declarative rule language, and propagate taint through assignments, arguments and return values.

6. Build a typed dependency graph and compute forward, backward and finding-focused slices, recording the reason for every edge and every statement included in a slice.
7. Present the artefacts and the findings through a browser workbench and an HTTP API, with export to JSON and to a self-contained HTML report.
8. Verify the implementation with an independent audit, a mutation check and a hand-labelled corpus, and publish the measured results in the documentation.

1.4 Scope of the Work

CT-C supports the scalar types int, float, char, bool and string, fixed-size array declarations, arithmetic and logical expressions, user-defined functions, conditionals and the while and for loops. The analysis assumes a single translation unit. Pointer arithmetic, type-punning through pointers, alias analysis, the preprocessor and separate compilation lie outside the language model, and the tool says so when it meets them rather than analysing a guess. Arrays are tracked as whole objects, so an element written on one path and read on another is treated conservatively. These boundaries are revisited in Section 5.6, where their effect on the interpretation of a finding is discussed.

1.5 Organisation of the Report

Chapter 2 reviews the literature that the implementation draws on and states the gap the project addresses. Chapter 3 describes the system design, the compiler phases and the rule language. Chapter 4 documents the implementation: the technology, the repository, the role of every file, a module-wise reference, a worked example and the diagnostics that qualify a run. Chapter 5 presents the working system through annotated screenshots, reports the verification and evaluation results, discusses their meaning, and closes with the current progress, the limitations and the planned work. References follow Chapter 5.

CHAPTER 2
LITERATURE SURVEY

2.1 Compiler Construction

The front end of CodeTrace follows the classical organisation described by Aho, Lam, Sethi and Ullman [1]: a scanner that recognises tokens, a parser that recognises phrases, and a semantic phase that binds names and checks types. The parsing method is the one introduced by Knuth [2], whose canonical LR(1) construction builds a deterministic automaton from item sets in which each item carries a lookahead symbol. The canonical automaton is precise but large, and two simplifications are standard. The SLR(1) method of DeRemer [3] replaces the item-specific lookaheads with FOLLOW sets, which keeps the table small at the cost of precision. The LALR(1) construction merges canonical states that share a core and computes the merged lookaheads, which recovers most of that precision in a table of the same size as SLR; DeRemer and Pennello give the efficient lookahead computation [11]. CodeTrace implements all three modes over one grammar, which turns a textbook distinction into something a reader can observe: the same productions produce 656 canonical states, 167 LALR states and 167 SLR states, with no conflicts in any mode.

Syntax-directed translation is also relevant beyond parsing. The abstract syntax tree used here follows the usual design in which each construct has a node class and each node records its source span, so that a diagnostic or a finding can name the exact text it concerns. The grammar of the rule language is processed by the same machinery, an application of the observation that a small declarative language is easier to validate with a generated parser than with hand-written string handling.

2.2 Data-Flow Analysis

Data-flow analysis computes, for every program point, a fact drawn from a lattice, by applying transfer functions along the edges of a control-flow graph until the results stop changing. Kildall's framework established that a broad class of optimisations can be expressed in this form and solved by iteration over a monotone framework [4]. Nielson, Nielson and Hankin present the same material as a theory of abstract interpretation, with explicit lattices, widening where the domain is infinite, and proofs that the iteration converges [5]. CodeTrace implements a single worklist solver with a bit-vector domain and instantiates it three times: reaching definitions as a forward may-analysis, liveness as a backward may-analysis, and definite assignment as a forward must-analysis. The solver records its iterations and its processing order, so the fixed point can be inspected rather than assumed.

2.3 Program Dependence and Slicing

Program slicing answers a narrower question than data-flow analysis: which statements can influence a chosen statement, or be influenced by it. Weiser introduced slicing as an aid to debugging and comprehension [6]. The program dependence graph of Ferrante, Ottenstein and Warren made the underlying relationships explicit by representing data and control dependencies as edges, which turns a slice into a graph reachability problem [7]. Interprocedural slicing extends the idea across function boundaries, and Horwitz, Reps and Binkley show how a system dependence graph supports it [12]; Reps, Horwitz and Sagiv give a graph-reachability formulation of precise interprocedural data flow under stated assumptions [8]. CodeTrace builds a typed dependency graph in which every edge records the analysis fact that justifies it, and computes slices by reachability over that graph. Each statement in a slice carries the reason it was included, which is the property the report relies on when it explains a finding.

2.4 Static Analysis for Security

Taint analysis adapts data-flow reasoning to a security question by distinguishing values that originate outside the program from values the program computes itself. Chess and McGraw describe the strengths and the persistent weaknesses of this approach: it scales to real code and finds classes of defect that testing misses, but its results depend on the sources and sinks it is told about and on the precision of the analysis [9]. Livshits and Lam demonstrate a practical taint analysis for Java in which sources, sinks and sanitizers are described declaratively, which keeps the engine independent of the rules [13]. Weakness catalogues, notably the Common Weakness Enumeration [10] and the OWASP Top 10 [14], supply the vocabulary in which such results are reported; the categories used by CodeTrace for command injection, SQL injection, path traversal and format strings correspond to CWE-78, CWE-89, CWE-22 and CWE-134.

2.5 Research Gap

Each of these traditions is well documented in isolation. What is less common, particularly in a course project, is a system in which the compiler artefacts and the security conclusion are visible at the same time and over the same source file. Commercial analysers present findings and hide the intermediate reasoning; textbooks present the algorithms and leave the integration to the reader. A student who wants to understand why a reported flow exists has to reconstruct the connection between the parser, the symbol table, the control-flow graph and the taint result, usually across several tools.

The second part of the gap concerns negative results. Tools that report nothing rarely explain what they examined. Since the meaning of an empty result depends on the language constructs, rules and approximations that were in force, a useful tool should state its own coverage: which input it rejected, which analysis limits it reached, and which sensitive calls it inspected without reporting. CodeTrace treats that statement as part of the result rather than as a footnote, and Section 3.6 describes where it appears.

The third part concerns reproducibility. Claims about a static analyser are easy to make and hard to check. The project addresses this by measuring its own claims: the audit script re-reads the documentation and the tree, the mutation check breaks the engine deliberately, and the corpus records an expectation before the analysis is run. A reader can repeat all three, which is the

property this report depends on.

Table 2.1: Summary of the literature and its use in the implementation.

Area	Principal sources	What CodeTrace takes from it
Scanning, parsing, syntax trees	Aho et al. [1]; Knuth [2]; DeRemer [3]; DeRemer and Pennello [11]	A hand-written scanner, canonical LR(1) item sets, LALR state merging, SLR fallback, and an abstract syntax tree with source spans.
Data-flow analysis	Kildall [4]; Nielson et al. [5]	A monotone worklist solver over a bit-vector lattice, instantiated as reaching definitions, liveness and definite assignment.
Dependence and slicing	Weiser [6]; Ferrante et al. [7]; Horwitz et al. [12]; Reps et al. [8]	A typed program dependence graph, and forward, backward and finding-focused slices computed by reachability.
Security analysis	Chess and McGraw [9]; Livshits and Lam [13]; CWE [10]; OWASP [14]	Declarative sources, sinks, sanitizers and validators; context-specific taint; findings reported in CWE terms with certainty and limitations.

2.6 Positioning of the Present Work

CodeTrace does not claim to replace a mature static application security testing product, and it does not claim detection rates that would make such a comparison meaningful. Its contribution is the integration itself: one small language, one grammar used for both the source language and the rule language, one pipeline whose intermediate results are all available to the reader, and a set of verification instruments that a third party can run. The supported language is deliberately narrow, and that narrowness is what makes a complete and honest description possible within a semester project.

CHAPTER 3
SYSTEM DESIGN

3.1 Architecture Overview

The system has two processes. The first is a Python server that serves the interface, exposes the HTTP API and runs the analysis; the second is the browser, which renders what the server returns. There is no build step, no bundler, no database and no network access beyond the server itself. The server is started with a single command, python3 -m codetrace.api, and the interface is then available at the same origin as the API, so the browser needs no knowledge of where it is running.

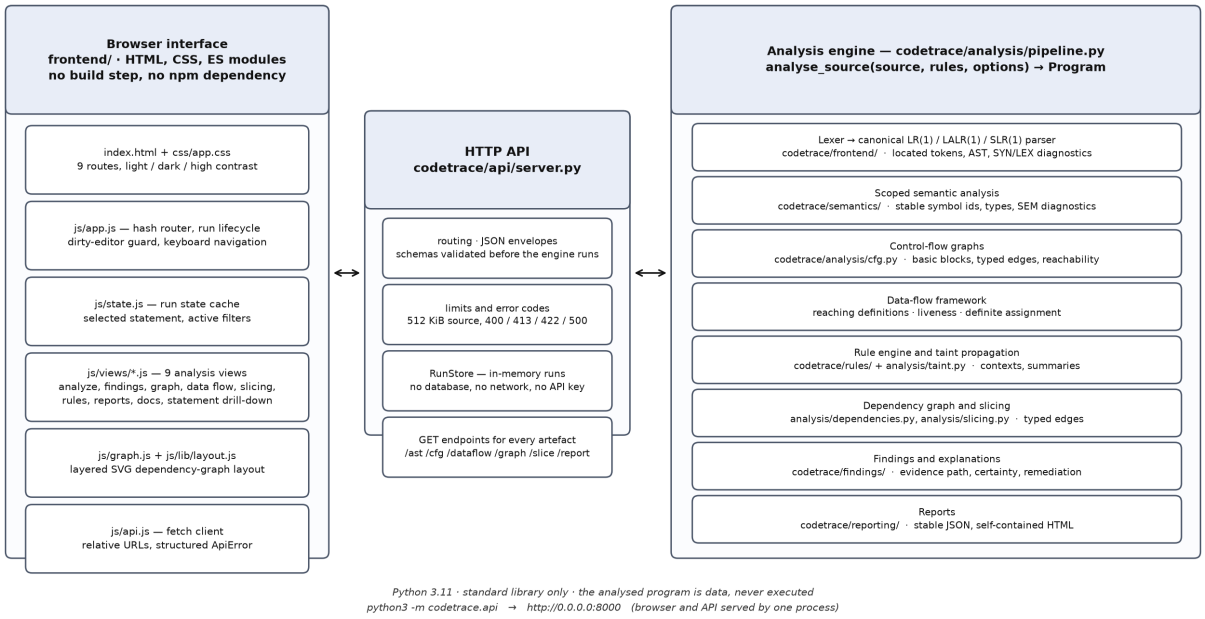


Figure 3.1: Runtime architecture: the browser workbench, the HTTP API and the analysis engine.

The separation is deliberate. The browser is a presentation layer: it displays structured results, keeps the current run in memory and requests detail on demand, but it never re-derives an analysis fact. This matters for the credibility of the screenshots in Chapter 5, because every number visible in the interface was produced by the engine and delivered over HTTP. The server, in turn, keeps recent runs in an in-memory store; it holds no persistent state, and it treats the submitted program as data.

3.2 Analysis Pipeline

A request enters the pipeline as source text and a set of options, and leaves it as a program object containing the intermediate representations, the diagnostics, the analysis results and the findings. The stages run in a fixed order, and each stage consumes the artefacts of the stage before it rather than re-reading the text.

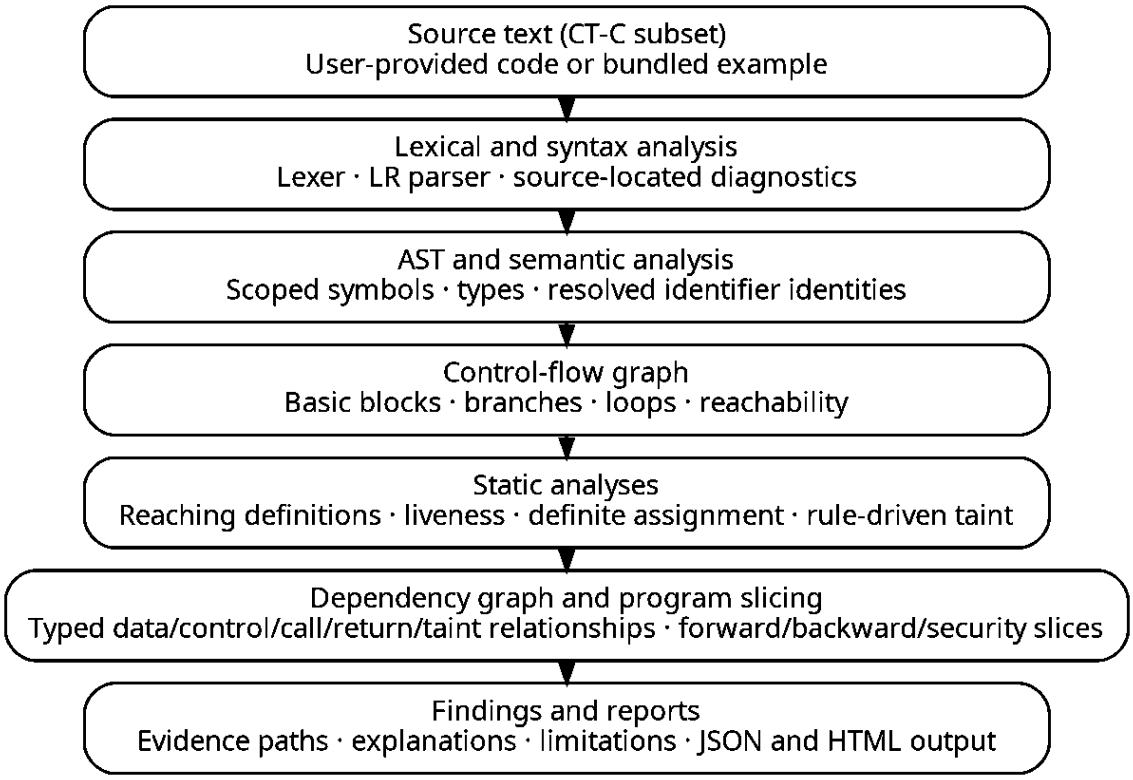


Figure 3.2: The analysis pipeline: each stage consumes the artefacts produced by the stage above it.

Table 3.1: Pipeline stages, their inputs and their outputs.

Stage	Input	Output
Lexical analysis	Source text	Tokens with offsets and line and column positions; lexical diagnostics
Syntax analysis	Token stream	Parse tree, parser statistics and syntax diagnostics
AST construction	Parse tree	Typed nodes carrying half-open source spans
Semantic analysis	AST	Scopes, symbols, call sites, per-statement reads and writes, semantic diagnostics
Control-flow graphs	AST and symbols	Basic blocks, typed edges, loops, reachability, numbered definitions
Data-flow analysis	CFGs	Reaching definitions, liveness, definite assignment and per-statement states
Taint analysis	CFGs, definitions, rules	Taint atoms, sink hits, function summaries, disposition records
Dependency graph	Statements, definitions, taint	Typed nodes and edges, each with a recorded reason
Slicing	Dependency graph	Statement sets with inclusion reasons and the induced subgraph
Findings and reports	All of the above	Findings with evidence paths, JSON and HTML reports

3.3 Module Description

The implementation is divided into packages with defined responsibilities. Each package operates on explicit data structures and publishes a small interface to the next, which is what allows the work-file tables in Section 4.4 to be read as a description of the design rather than as an inventory.

Table 3.2: Package responsibilities and principal outputs.

Package	Responsibility	Principal output
codetrace/frontend	Tokenisation, grammar representation, LR automata and tables, parser, AST definitions, grammar documentation.	Tokens, syntax tree, parser statistics, syntax diagnostics.
codetrace/semantics	Scope handling, symbol identity, type and declaration checks, per-statement accesses.	Resolved symbols, call sites, semantic diagnostics.
codetrace/analysis	Control-flow graphs, the data-flow framework and its three instances, taint propagation, dependency graph, slicing, pipeline orchestration.	CFGs, data-flow states, taint facts, dependency graph, slices, the program object.
codetrace/rules	Rule model and validation, the declarative rule language and its parser, the built-in rule set.	An immutable rule set with a content fingerprint.
codetrace/findings	Finding model, stable identity, comparison of runs, evidence path and certainty.	Findings with explanations and remediation guidance.
codetrace/reporting	Serialisation of a run.	A stable JSON report and a self-contained HTML report.
codetrace/api	HTTP routing, request validation, limits, the run store and the schema catalogue.	A JSON API and the static interface.
frontend	The nine views, the graph renderer, the state store and the stylesheet.	The workbench shown in Chapter 5.

3.4 Compiler Phases

The phases of the pipeline are the classical ones, but each is implemented rather than assumed, and each can be inspected from the interface.

3.4.1 Lexical Analysis

The scanner is deterministic and advances by at least one character on every iteration, so it cannot loop. It skips whitespace and both comment forms, recognises identifiers, decimal and hexadecimal integers, floating-point and character literals, string literals with escape sequences including hexadecimal escapes, punctuation and operators, and records for every token a byte offset plus a one-based line and column. Malformed or unsupported input is reported at the position where it occurs: LEX100 for a construct outside the subset such as switch, LEX101 for a preprocessor directive, LEX102 for a stray character. The token stream and the accumulated statistics are available through the API, which is how the numbers quoted in this report can be checked.

3.4.2 Syntax Analysis and Parser Generation

The parser is generated at start-up from a grammar written for CT-C. The generator computes the nullable and first sets, builds the canonical LR(1) item sets with closure and goto, and can either use those states directly, merge states with a common core to obtain LALR(1), or fall back to FOLLOW-based lookaheads for SLR(1). A conflict is recorded with the state and the symbol that produced it rather than resolved silently, and the tables are cached per mode so that repeated analyses do not rebuild them.

Table 3.3: Table construction for the same grammar in the three supported modes.

Mode	States	Productions	Conflicts	Build time
LALR(1), the default	167	102	0	0.78 s
Canonical LR(1)	656	102	0	0.87 s
SLR(1)	167	102	0	0.06 s

Measured with codetrace.frontend.pipeline.parser_tables on this checkout. The canonical automaton is built before merging, which is why it has 656 states against 167.

Recovery is panic mode with a viable-prefix test. When an unexpected token is found the parser discards input until the stack, the lookahead and the remaining input admit a continuation, and it reports the token it expected. The run is then marked partial, as Section 4.7 illustrates, so a recovered file is never presented as a clean analysis.

3.4.3 Semantic Analysis

The semantic phase walks the tree with a stack of scopes. Function parameters enter a scope of their own, compound statements open nested scopes, and every declaration receives a stable integer identifier that is never reused within a run. Later stages therefore refer to identifier 7 rather than to the name x, which is what makes shadowing harmless: an inner declaration is a different symbol, and a read resolves to the symbol that the scope stack selects at that point. The phase reports undeclared names, duplicate declarations, type mismatches, wrong argument counts and calls to unknown functions, and it records the reads and writes of every statement for the data-flow problems to consume.

3.4.4 Control-Flow Graphs

Each function is converted into basic blocks linked by typed edges: sequential, true, false, loop-back, break, continue and return. Block identifiers are stable within a run, loop headers and back edges are identified, and reachability is computed from the entry block by breadth-first search, so a block is described as unreachable only when that search could not reach it. Definitions are numbered once in a pre-pass and shared by every analysis that follows.

3.4.5 Data-Flow Analysis

The framework exposes a problem as a direction, a domain, a boundary value, transfer functions, a meet operation and optional iteration caps. The solver is a worklist algorithm over bit vectors, and it returns the fixed point together with the number of iterations, the number of steps and the order in which blocks were processed. Three problems are instantiated: reaching definitions as a forward may-analysis, liveness as a backward may-analysis and definite assignment as a forward must-analysis. The last of these produces a diagnostic when a variable may be read before it is written, which is reported as a warning rather than as a finding, because it is a property of the program rather than a security conclusion.

3.4.6 Taint Propagation

Taint is represented as a set of atoms of the form src:rule@context. A value carries one atom per source rule and per context in which it is untrusted, so escaping a string for SQL does not remove its command context, and parsing it into an integer removes every context at once. The taint problem runs as another instance of the data-flow framework, augmented with a call graph: arguments are matched to parameters, return values are propagated to call sites, and the summaries are iterated until they stop changing. The approximation this implies is stated in every finding that crosses a function boundary: the summary is context-insensitive, so a flow reported through a function may be a path that no single execution can realise.

3.4.7 Dependency Graph and Slicing

The dependency graph connects statements and definitions with typed edges for data, control, call, return, definition, sanitisation, taint and source-to-sink relations. Every edge carries the analysis fact that justifies it: a reaching definition, a live variable, a call summary or a taint atom. Slicing is then reachability over that graph, in the direction the reader chooses, and each statement in the resulting slice keeps the reasons it was reached. A slice is conservative by construction; it contains every statement the analysis cannot exclude, and it is not claimed to be minimal.

3.5 Rule Language Design

Sources, sinks, sanitizers, validators and flows are declared in a small textual language rather than in code. A rule file is parsed by the same LR machinery as CT-C, validated with line and column diagnostics, and only then converted into an immutable rule set with a fingerprint over its content. The fingerprint is displayed by the interface and recorded in every report, so a result can be attributed to the rule set that produced it. Because a rule file is untrusted input, the language has no expressions, no control flow and no access to the host.

Rule declarations, from codetrace/rules/builtin/standard.ctrule.

```
version 1

source stdin {
    description "one line read from standard input"
    function input
    returns
    contexts command, sql, path, html, numeric, format
}

sink command_system {
    description "executes its argument through the system command interprete
r"
    function system
    arguments 0
    context command
}

flow command_execution {
    sources stdin
    sinks command_system
    sanitizers shell_escape, parse_to_int
    severity HIGH
    category "command-injection"
}
```

Table 3.4: The built-in rule set.

Kind	Count	Declared identifiers
Sources	5	stdin, stdin_int, environment, command_line, file_content
Sinks	8	command_system, command_exec, command_popen, path_fopen, path_readfile, path_writefile, sql_exec, format_printf
Sanitizers	5	shell_escape, sql_escape, html_escape, path_normalize, parse_to_int
Validators	2	validate_command, validate_path
Flows	4	command_execution, sql_injection, path_traversal, format_string

Fingerprint of this set: builtin-3387b7dcd97a95f3.

3.6 Data Model and Interface Contract

An analysis produces a program object, and the API exposes each part of it at its own endpoint. The interface contract is documented in docs/API.md and described here because the views in Chapter 5 are, in effect, presentations of these responses. Requests are validated before the engine runs, and an invalid request receives a structured error envelope naming the field and the reason.

Table 3.5: Principal HTTP endpoints.

Endpoint	Returns
POST /api/analyze	Runs the pipeline over a submitted file and returns the summary, the metrics and the findings.
POST /api/rules/validate	Parses and validates a rule file, with line and column diagnostics.
POST /api/compare	Compares two runs and classifies findings as new, resolved, persistent or moved.
GET /api/runs/{id}/findings	Findings with filter and search parameters, each with its evidence path.
GET /api/runs/{id}/findings/{fid}/slice	The security slice for one finding, with inclusion reasons.
GET /api/runs/{id}/ast	The syntax tree with source spans, symbol identifiers and resolutions.
GET /api/runs/{id}/cfg	One function's control-flow graph, or every graph at depth one with statistics.
GET /api/runs/{id}/dataflow	The chosen analysis for one function: sets, per-statement states, iterations.
GET /api/runs/{id}/graph	The dependency graph with typed edges and the reason behind each edge.
GET /api/runs/{id}/statements/{sid}	One statement with its definitions, live variables, taint atoms and dependencies.
GET /api/runs/{id}/report	The exported report, as JSON or as a self-contained HTML document.
GET /api/health, /api/grammar, /api/language, /api/rules, /api/examples	Version, grammar statistics, language description, rule catalogue and example index.

3.7 Interface Design

The workbench is organised as nine views over a single run, reached from a fixed navigation column. Every view reads the same run identifier, so the artefacts cannot disagree about which analysis is being displayed. Tables are used where the data is tabular, and a computed layered layout is used for graphs; the renderer is written for the project so that node positions are deterministic and so that graph nodes remain focusable from the keyboard.

Table 3.6: The nine views and their purpose.

View	Purpose
Overview	Engine and rule-set identity, the summary of the current run, and the pipeline that produced it.
Analyze Code	Source input, parser mode, rule options, run summary and the diagnostics for the submitted file.
Findings	Filterable findings with the evidence path, certainty, limitations and remediation for the selected finding.
Program Graph	The typed dependency graph, the control-flow graph and the call view, with edge filters and a statement inspector.
Data-Flow Analysis	Per-block and per-statement states for reaching definitions, liveness or definite assignment.
Program Slicing	Backward, forward and finding-focused slices with the reason each statement was included.
Rule Editor	Validation of a rule file with located diagnostics, and the catalogue of rules in force.
Reports	Export as JSON or HTML, and comparison of two runs by stable finding identity.
Documentation	The repository's documents, the supported language and the diagnostics catalogue.

3.8 Design Decisions and Trade-offs

Several decisions shaped the implementation, and each has a cost that the report states rather than conceals.

Table 3.7: Principal design decisions and their consequences.

Decision	Rationale	Cost
Standard library only for the engine	The examiner can run the work without installing anything, and every algorithm remains visible in the repository.	No third-party solver or parsing library; the data-flow machinery is written and reviewed in-house.
One grammar used for source and rule language	A single parsing mechanism, and rule files are validated by the same machinery that validates CT-C.	The rule language is deliberately inexpressive: declarations only, no expressions.
Immutable rule sets with a fingerprint	A result can be attributed to the exact rules that produced it.	Rules cannot be edited in place during a run.
In-memory run store	No database, no migration, no persistence to secure.	Runs do not survive a restart; reports are generated on demand.
Conservative analysis	A reported flow should correspond to a path the program can take, at the price of missing some flows.	False negatives remain, and are documented per finding rather than hidden.
Explicit partial-analysis status	A recovered or limited run is distinguishable from a complete one.	The reader must attend to the status before quoting a result.

CHAPTER 4
IMPLEMENTATION DETAILS

4.1 Technology Stack

The engine is written in Python 3.11 and uses only the standard library. The interface is written in HTML, CSS and native ECMAScript modules, without a bundler and without a runtime dependency. The choice keeps the installation story to one command and keeps the algorithms in a form that can be read alongside the theory that motivates them.

Table 4.1: Technology choices and the reasons behind them.

Layer	Technology	Reason
Engine	Python 3.11, standard library only	No installation step; the algorithms are readable and auditable.
Parser generation	Canonical LR(1) with LALR merging and SLR mode	One grammar exercises three table constructions that the course theory distinguishes.
Rule language	Declarative text, parsed by the same LR machinery	Rules are validated with located diagnostics before any analysis runs.
Interface	HTML, CSS, native ES modules	No build step; module boundaries stay explicit.
Graph rendering	Layered layout implemented for the project	Deterministic positions, keyboard-reachable nodes, no layout library.
Transport	JSON over HTTP on a single origin	Relative URLs, so the interface also works behind a forwarded port.
Persistence	In-memory run store	The tool is local and stateless; a database would add cost without benefit.
Report export	Stable JSON and a self-contained HTML document	The result can be reviewed offline, and all analysed text is escaped.

4.2 Repository Organisation

The complete implementation, the tests, the labelled corpus, the screenshots and the documentation are held in one public repository, https://github.com/lostpilot404/CD-Project. The default branch contains the state described in this report.

Table 4.2: Repository layout, with file and line counts measured from this checkout.

Directory	Files	Lines	Contents
Compiler front end (codetrace/frontend)	9	3,277	Lexer, grammar, LR automata and tables, parser, syntax tree, grammar notes
Semantic analysis (codetrace/semantics)	4	1,352	Scopes, symbol identities, type and declaration checks, call sites
Rule language (codetrace/rules)	4	1,235	Rule parser, immutable rule set and the built-in rule file
Program analyses (codetrace/analysis)	11	4,130	Control-flow graphs, data-flow framework and instances, taint, dependency graph, slicing, pipeline
Findings and reporting (codetrace/findings, codetrace/reporting)	6	1,875	Finding model, explanations and evidence paths, JSON and HTML reports
HTTP API (codetrace/api)	4	1,152	Server, routing, request validation, schema and catalogue endpoints
Package root (codetrace/)	2	22	Version constants and the package docstring
Interface (frontend/)	21	4,350	ES-module workbench: nine views, graph rendering, styles and the HTML shell
Interface tests (frontend/tests)	5	2,115	Stub DOM, test harness, library and view tests
Engine tests (tests/)	18	6,212	Lexer, parser, semantics, control flow, data flow, taint, slicing, rules, API, reporting, workflow, audit
Corpus and harness (evaluation/)	2	754	Expectations for the labelled corpus and the harness that runs it
Tooling (scripts/)	16	9,300	Server, test, evaluation, audit, screenshot and report scripts
Example programs (examples/)	34	307	The 33 labelled CT-C programs and one example rule file
Documentation (docs/)	13	4,050	Design notes, guides, the project report and the screenshot sets

Total: 149 files and 40,131 lines across the directories listed, including the built-in rule file and this report's builder but excluding binary assets. The module table in `docs/TECHNICAL_DESIGN.md` *counts Python sources only, so its engine total is one file and 209 lines lower;* `frontend/tests` *is listed here under its own heading and is not counted again inside* `frontend/`.

Three conventions make the repository navigable. Every package carries a docstring that states its responsibility, so the work-file tables in Section 4.4 can quote the code rather than paraphrase it. Every claim in the documentation has a command behind it, collected in the README and in Section 4.8. And every deliverable is generated: the reports, the screenshots and the figures are produced by scripts in the repository, so a change in the engine can be carried through to the

documents by rebuilding them.

4.3 Implementation Roadmap and Review 3 Deliverables

The work proceeded in three review cycles. Review 1 established the language, the lexer, the parser and the semantic phase. Review 2 completed the analysis pipeline and the workbench. Review 3 concentrated on demonstration and on verification: the interface was exercised end to end and captured, the documentation was reorganised, and the claims were re-measured against the tree.

Table 4.3: Deliverables by review cycle.

Cycle	Delivered	Evidence in this report
Review 1	Language definition, lexer, LR tables, parser, AST, scoped semantic analysis.	Sections 3.4.1 to 3.4.3; Tables 3.2 and 3.3
Review 2	Control-flow graphs, data-flow framework and its three instances, rule language, taint analysis, dependency graph, slicing, HTTP API, nine-view workbench, JSON and HTML reports.	Sections 3.4.4 to 3.6 and 3.8; Table 3.6
Review 3	Complete screenshot set captured from the running system, report restructured for review, module reference and work-file documentation, verification and evaluation results re-measured, interface defect in the editor-state warning corrected.	Chapter 5; Tables 5.1 to 5.4; Figures 5.1 to 5.16

The interface defect corrected in this cycle is worth recording because it illustrates the value of capturing the system rather than describing it. The warning that appears when the editor has been modified after a run was rendered as an inline element inside the status line, so its text overflowed the column and overlapped the panel beneath it. The screenshot in Figure 5.3 is the evidence and the corrected rendering; the change itself is a container element and a wrapping rule in the stylesheet, with no effect on the analysis.

4.4 Work File Explanation

The tables in this section describe every file in the repository, grouped by subsystem. The purpose of a source file is quoted from the docstring of the module itself where one exists, so the description cannot drift away from the code without the difference becoming visible. Line counts are measured as the file stands in this checkout, and binary assets are excluded.

4.4.1 Compiler front end (codetrace/frontend)

Table 4.4: Compiler front end (codetrace/frontend): 9 files, 3,277 lines.

File	Lines	Purpose
__init__.py	34	Front end for the CT-C source subset.
ast_nodes.py	431	Abstract syntax tree for the CT-C subset.
clr_parser.py	651	Table-driven LR parser: SLR(1), LALR(1) and canonical LR(1).
grammar.py	493	Grammar representation and LR analysis primitives.
grammar_lang.py	541	The CT-C grammar: productions, semantic actions and grammar documentation.
lexer.py	689	Hand-written lexer for the CT-C source subset.
pipeline.py	129	Lex + parse entry point.
source.py	43	Source-text helpers shared by the front end and the reporting layer.
tokens.py	266	Token definitions for the CodeTrace source language (CT-C).

Paths are relative to `codetrace/frontend/`.

4.4.2 Semantic analysis (codetrace/semantics)

Table 4.5: Semantic analysis (codetrace/semantics): 4 files, 1,352 lines.

File	Lines	Purpose
__init__.py	0	Package marker for semantic analysis.
analyzer.py	913	Semantic analysis: declaration collection, scope resolution, type checking.
externals.py	196	Model of the host environment: known external functions.
symbol_table.py	243	Symbol tables with stable symbol identities.

Paths are relative to `codetrace/semantics/`.

4.4.3 Rule language (codetrace/rules)

Table 4.6: Rule language (codetrace/rules): 4 files, 1,235 lines.

File	Lines	Purpose
__init__.py	0	Package marker for the rule model.
builtin/standard.ctrule	209	The built-in rule set: five sources, eight sinks, five sanitizers, two validators and four flows.
model.py	305	Security rule model and registry.
rule_language.py	721	Grammar, parser and validator for the CodeTrace security rule language.

Paths are relative to `codetrace/rules/`.

4.4.4 Program analyses (codetrace/analysis)

Table 4.7: Program analyses (codetrace/analysis): 11 files, 4,130 lines.

File	Lines	Purpose
__init__.py	0	Package marker for the analysis stages.
accesses.py	153	Variable reads and writes per statement.
cfg.py	646	Control-flow graph construction over the CT-C AST.
dataflow.py	324	A small, reusable data-flow analysis framework.
definite_assignment.py	172	Definite-assignment analysis (the "possibly uninitialised variable" check).
dependencies.py	547	Program dependency graph: data, control, call, return, taint and sanitizer edges.
liveness.py	159	Live-variable analysis.
pipeline.py	536	Whole-program analysis pipeline.
reaching_definitions.py	185	Reaching-definitions analysis.
slicing.py	325	Program slicing over the program dependency graph.
taint.py	1,083	Taint analysis: sources, sinks, sanitizers, interprocedural propagation.

Paths are relative to `codetrace/analysis/`.

4.4.5 Findings and reporting (codetrace/findings, codetrace/reporting)

Table 4.8: Findings and reporting (codetrace/findings, codetrace/reporting): 6 files, 1,875 lines.

File	Lines	Purpose
codetrace/findings/__init__.py	0	Package marker for the finding model.
codetrace/findings/explain.py	1,026	Finding construction and path explanation.
codetrace/findings/models.py	199	Finding model.
codetrace/reporting/__init__.py	0	Package marker for the report builders.
codetrace/reporting/html_report.py	297	Human-readable HTML report.
codetrace/reporting/json_report.py	353	Analysis report: the documented, stable JSON structure CodeTrace exports.

Paths are relative to the repository root.

4.4.6 HTTP API (codetrace/api)

Table 4.9: HTTP API (codetrace/api): 4 files, 1,152 lines.

File	Lines	Purpose
__init__.py	0	Package marker for the HTTP layer.
__main__.py	21	Run the CodeTrace HTTP API: python3 -m codetrace.api [options].
schemas.py	140	Request/response schemas of the CodeTrace HTTP API.
server.py	991	CodeTrace HTTP API (Python standard library only).

Paths are relative to `codetrace/api/`.

4.4.7 Package root (codetrace/)

Table 4.10: Package root (codetrace/): 2 files, 22 lines.

File	Lines	Purpose
codetrace/__init__.py	0	Package marker; the version constants live in version.py.
codetrace/version.py	22	Single source of truth for the CodeTrace version identifiers.

Paths are relative to the repository root.

4.4.8 Interface (frontend/)

Table 4.11: Interface (frontend/): 21 files, 4,350 lines.

File	Lines	Purpose
css/app.css	1,320	All styling: layout, tables, graph canvas, and the light, dark and high-contrast themes.
index.html	201	Single-page shell holding the navigation, the nine view containers and the analysis form.
js/api.js	94	HTTP client with relative URLs, request timeouts and structured error values.
js/app.js	632	CodeTrace interface: routing, state wiring and view updates.
js/graph.js	176	SVG renderer for the program dependency graph, the CFG and slice views.
js/lib/dom.js	65	Minimal DOM helpers.
js/lib/filter.js	106	Filtering, searching and sorting of findings.
js/lib/layout.js	117	Deterministic layered layout for program graphs.
js/source.js	83	Renders the analysed source with statement highlighting and line mapping.
js/state.js	106	Application state.
js/views/analyze.js	258	Analyze Code: submit source to the engine and show what came back.
js/views/dataflow.js	129	Data-Flow Analysis view: the CFG of one function with the results of the classical analyses, exactly as the engine computed them.
js/views/docs.js	86	Documentation view: the language subset, the diagnostic catalogue, the grammar of the analysed language and the project documents.
js/views/findings.js	272	Findings: the list, the filters and the evidence for one finding.
js/views/graph.js	286	Program Graph view: the dependency graph, a single CFG, or the subgraph of a finding / slice.
js/views/overview.js	114	Overview: what the tool is, what it just analysed, and how the pipeline fits together.
js/views/reports.js	51	Reports: export the JSON or HTML report of the current run, inspect the schema and compare two runs (new / resolved / persistent / moved findings).
js/views/rules.js	77	Rule Editor.
js/views/slicing.js	125	Program Slicing view.
js/views/statement.js	42	Selected-statement panel, shared by the graph, data-flow and slicing views.

File	Lines	Purpose
package.json	10	Test-only manifest; the application itself has no npm dependency.

Paths are relative to `frontend/`.

4.4.9 Interface tests (frontend/tests)

Table 4.12: Interface tests (frontend/tests): 5 files, 2,115 lines.

File	Lines	Purpose
dom.js	418	A deliberately small DOM implementation for the frontend tests.
harness.js	150	Test harness: a real engine plus a stub DOM.
lib.test.js	337	Unit tests for the frontend's pure modules: DOM helpers (escaping), finding filters, graph layout and the SVG renderer's edge selection.
regressions.test.js	377	Regression tests for the interface defects confirmed by the hostile audit.
views.test.js	833	Interface integration tests.

Paths are relative to `frontend/tests/`.

4.4.10 Engine tests (tests/)

Table 4.13: Engine tests (tests/): 18 files, 6,212 lines.

File	Lines	Purpose
__init__.py	1	CodeTrace test suite (Python standard library unittest).
helpers.py	69	Shared helpers for the test suite.
test_api.py	687	API tests against a real HTTP server.
test_audit.py	187	Tests for the audit script.
test_audit_regressions.py	668	Regression tests for compiler, HTTP API and interface defects.
test_cfg.py	394	Control-flow graph tests.
test_clr_theory.py	294	Parser-generator checks against textbook answers and an independent builder.
test_core_battery.py	123	Hand-labelled taint battery (positive, negative and boundary cases).
test_dataflow.py	708	Data-flow analysis tests.
test_lexer.py	124	Lexer tests: tokens, positions, literals, comments and every lexer diagnostic.
test_mutation_check.py	132	Tests for the mutation checker itself.
test_parser.py	300	Parser tests: LR(1) construction, parsing, diagnostics and recovery.
test_reporting.py	393	Reporting tests: the JSON report and the self-contained HTML report.
test_rules.py	353	Rule language tests: the declarative rule file is untrusted input.
test_semantics.py	164	Semantic analysis tests: scopes, shadowing, types, symbols and diagnostics.
test_slicing.py	289	Dependency graph and slicing tests.
test_taint.py	779	Taint analysis tests: the flows that must be found and the ones that must not.
test_workflow.py	547	End-to-end workflow tests over the real HTTP API.

Paths are relative to `tests/`.

4.4.11 Corpus and harness (evaluation/)

Table 4.14: Corpus and harness (evaluation/): 2 files, 754 lines.

File	Lines	Purpose
expectations.json	420	The hand-written labels for the 33-program corpus, one record per case.
run_evaluation.py	334	Evaluate CodeTrace against the labelled corpus in evaluation/expectations.json.

Paths are relative to `evaluation/`.

4.4.12 Tooling (scripts/)

Table 4.15: Tooling (scripts/): 16 files, 9,300 lines.

File	Lines	Purpose
audit.py	1,007	Independent audit: re-derives the claims the documents make from the tree and the engine.
build_project_report.py	775	Builds docs/CodeTrace_Project_Report.pdf from its editable Markdown source.
build_review2_docx.py	1,004	Builds the Review 2 report; kept so that the earlier submission can be rebuilt.
build_review3_docx.py	1,183	Layout of this report: template pages, typography, tables, figures and measured page numbers.
capture_review2_screenshots.py	144	Captures the Review 2 screenshots; kept with the report it produced.
capture_screenshots.js	350	Drives the workbench in a headless browser and captures the sixteen screenshots of Chapter 5.
evaluate.sh	19	Runs the labelled corpus and writes JSON evidence under var/.
generate_review2_pdf.py	1,023	Generates the Review 2 PDF preview; kept with the report it produced.
make_review2_figures.py	219	Builds the figure assets used by the Review 2 report.
make_review3_figures.py	198	Draws Figure 4.2 from the dependency graph the engine computes for the worked example.
mutation_check.py	401	Injects deliberate defects into the engine and checks that the suite notices each one.
review2_content.py	1,239	Report content for the Review 2 Word report.
review3_content.py	1,577	Wording of this report: every chapter, section, table row and caption.
serve.sh	18	Wrapper that starts the server on a chosen port.
tests.sh	41	Runs every suite: engine, API, reporting, workflow and interface.
trim_screenshots.py	102	Crops the empty tail from a capture and scales it to the resolution the page needs.

Paths are relative to `scripts/`.

4.4.13 Example programs (examples/)

Table 4.16: Example programs (examples/): 34 files, 307 lines.

File	Lines	Purpose
challenges/array_element_taint.ctc	8	Expected 1 finding: taint on an array element is attributed to the whole array.
challenges/decoy_in_comment_and_string.ctc	9	Expected 1 finding only: the two decoys below are text, not calls.
challenges/guarded_and_checked.ctc	8	Expected 1 finding, feasibility guarded: the sink is inside a branch that was not evaluated.
challenges/loop_carried.ctc	11	Expected 1 finding: the tainted value flows through a loop-carried assignment.
challenges/multiple_sources_two_sinks.ctc	8	Expected 2 findings: stdin and an environment variable reach two different sinks.
challenges/negated_validator_guard.ctc	8	Expected 1 finding, feasibility guarded: the validator guards the other branch.
challenges/shadowed_variable.ctc	10	Expected 1 finding: only the outer name is tainted; the inner one is a constant.
challenges/unknown_external_keeps_taint.ctc	7	Expected 1 finding plus a SEM008 warning: an unknown function never clears taint.
challenges/wrong_context_sanitizer.ctc	8	Expected 2 findings: sql_escape does not neutralise command context, and the raw value still reaches sql_exec.
custom_rules/custom_rule_demo.ctc	9	Example: used with examples/custom_rules/strict_rules.ctrule: the custom sink is.
custom_rules/strict_rules.ctrule	36	Example rule file used by the rule-editor demonstration in Chapter 5.
interprocedural/argument_flow.ctc	15	Expected 1 finding, rule command_execution: the taint crosses two call boundaries.
interprocedural/recursion.ctc	14	Expected 1 finding, rule command_execution, certainty unproven (recursion limit reached).
interprocedural/return_value.ctc	11	Expected 1 finding, rule command_execution, interprocedural true.
safe/constant_command.ctc	6	Expected no findings, no diagnostics.
safe/dead_branch_sink.ctc	8	Expected no findings: the sink sits in a branch whose condition is the literal 0.
safe/killed_by_constant.ctc	7	Expected no finding: the tainted value is overwritten before the sink.
safe/source_never_reaches_sink.ctc	7	Expected no findings: the source value is overwritten before the sink.

File	Lines	Purpose
safe/unrelated_variable.ctc	8	Expected no findings: the tainted value is printed, but a different value reaches the sink.
sanitized/parse_to_int.ctc	8	Expected no findings: to_int destroys the text representation of the value.
sanitized/shell_escape.ctc	7	Expected no findings: shell_escape neutralises command-context taint.
sanitized/sql_escape.ctc	7	Expected no findings: sql_escape neutralises SQL-context taint.
sanitized/validated_command.ctc	8	Expected no findings: the value is used only behind a declared validator.
syntax_errors/broken_expression.ctc	5	Expected a syntax error inside the expression, reported at the failing token.
syntax_errors/missing_semicolon.ctc	6	Expected exactly one SYN001 error with a line:column and an expected-token list.
syntax_errors/unbalanced_braces.ctc	5	Expected a syntax error, and the parser reports where the construct is incomplete.
unsupported/pointer_type.ctc	6	Expected syntax/unsupported diagnostics: pointer types are outside the analysed subset.
unsupported/preprocessor_directive.ctc	7	Expected LEX101 preprocessor diagnostic; the directive is reported, not ignored.
unsupported/switch_statement.ctc	9	Expected syntax/unsupported diagnostics; the file is not analysed as if switch were safe.
vulnerable/command_injection.ctc	12	Expected 1 finding, rule command_execution, HIGH, category command-injection.
vulnerable/format_string.ctc	6	Expected 1 finding, rule format_string, MEDIUM.
vulnerable/path_traversal.ctc	8	Expected 1 finding, rule path_traversal, HIGH.
vulnerable/sql_injection.ctc	7	Expected 1 finding, rule sql_injection, CRITICAL, category sql-injection.
vulnerable/two_step_command.ctc	8	Expected 1 finding, rule command_execution, reached through three assignments.

Paths are relative to `examples/`.

4.4.14 Documentation (docs/)

Table 4.17: Documentation (docs/): 13 files, 4,050 lines.

File	Lines	Purpose
ALGORITHMS.md	438	Pseudocode, complexity and measured timings for every algorithm in the engine.
API.md	233	Every endpoint, its request and response schema, the error envelope and the limits.
COMPILER_THEORY.md	901	The compiler theory the implementation rests on, with definitions and worked examples.
CodeTrace_Project_Report.md	358	Editable source of the BCSE307P project report, including its verification evidence.
EVALUATION.md	247	How the labelled corpus was built and what the reported measurements mean.
LANGUAGE.md	288	The CT-C subset: tokens, grammar, types, statements and every diagnostic code.
LIMITATIONS.md	177	What the analysis does not do, and the threat model of the tool itself.
PRESENTATION.md	295	Talk outline, demonstration script and viva questions.
REPORT_SCHEMA.md	154	The exported report: keys, finding fields, path steps and stability rules.
REVIEW_2_REPORT.md	175	Review 2 deliverable summary: features, work-file map, module reference and measured results.
RULE_LANGUAGE.md	294	The declarative rule language: grammar, attributes, validation rules and examples.
TECHNICAL_DESIGN.md	249	Architecture, module responsibilities, the data flow through the engine and design decisions.
USER_GUIDE.md	241	Running the application, reading a finding and the fix-and-recheck loop.

Paths are relative to `docs/`.

4.5 Module-Wise Reference

The reference below is intended for a reader who wants to check one claim quickly. For each module it names the principal symbols, states the invariant the module maintains and points to the part of the literature it implements. Reference numbers correspond to the bibliography.

4.5.1 Front End and Semantic Analysis

Table 4.18: Front-end and semantic modules.

Module and principal symbols	Invariant and output	Ref.
frontend/lexer.py: Lexer, tokenize, Diagnostic, LexResult	Every token has a kind, a value, a byte span and a line and column; the scanner always advances, and every lexical error carries a code.	[1], [2]
frontend/tokens.py: Token, display_kind	Token kinds are stable strings, so diagnostics, tests and documentation refer to the same names.	[1]
frontend/grammar.py: Grammar, Production, Automaton, build_automaton	Nullable and first sets, closure and goto, canonical LR(1) items, LALR merging and SLR lookaheads over one augmented grammar.	[1], [2], [3], [11]
frontend/clr_parser.py: build_tables, ParserTables, LRParser, Action	One table row per state; a conflict is recorded with its state and symbol, never hidden; recovery preserves the longest viable prefix.	[2], [3], [11]
frontend/grammar_lang.py: LanguageGrammarBuilder, grammar_documentation	The CT-C grammar is written once, and the documentation is generated from the same description.	[1]
frontend/ast_nodes.py: Node and one class per construct	Every node carries a half-open source range, and traversal order is deterministic.	[1]
frontend/pipeline.py: parse_source, parser_tables, ParseOutcome	Tables are built once per mode and cached; the outcome carries diagnostics and an optional tree.	[2]
frontend/source.py: LineIndex, split_lines	Byte offsets and line and column positions are converted without re-scanning the file.	—
semantics/symbol_table.py: Symbol, Scope, SymbolTable, Reference	Every declaration receives a stable identifier; scopes nest and shadowing is explicit.	[1], [5]
semantics/analyzer.py: SemanticAnalyzer, analyse, SemanticResult	Names resolve to symbols rather than to strings; reads and writes are recorded per statement for the analyses downstream.	[1], [5]
semantics/externals.py: ExternalFunction, lookup, catalog_json	The external-function catalogue is data, so a rule and a semantic check agree on arity and category.	[1]

4.5.2 Control Flow, Data Flow and Taint

Table 4.19: Analysis modules.

Module and principal symbols	Invariant and output	Ref.
analysis/cfg.py: CFG, BasicBlock, CFGEdge, Definition, build_cfgs	Blocks are maximal straight-line sequences; edges are typed; definitions are numbered once and shared.	[4], [5]
analysis/accesses.py: statement_accesses, expression_accesses	Reads and writes are computed once, so GEN and KILL agree with USE and DEF.	[5]
analysis/dataflow.py: DataFlowProblem, BitVectorDomain, solve	One monotone worklist solver with documented iteration and step caps; IN and OUT are bit vectors over a fixed domain.	[4], [5]
analysis/reaching_definitions.py: ReachingDefinitions, definition_atom	Forward may-analysis answering which definitions may reach a statement.	[4], [5]
analysis/liveness.py: Liveness, symbol_atom	Backward may-analysis answering which values may be read later.	[4], [5]
analysis/definite_assignment.py: DefiniteAssignment	Forward must-analysis; a read that may precede a write is reported as DFA001.	[4], [5]
analysis/taint.py: TaintEngine, TaintProblem, SinkHit, TaintResult	One atom per context; a sanitizer removes only its own context; summaries iterate to a fixed point and their approximation is recorded.	[9], [10], [13]
analysis/dependencies.py: DependencyGraph, DepEdge, DepNode	Every edge names the analysis fact that justifies it; no edge is added without a reason.	[7], [12]
analysis/slicing.py: SliceEngine, Slice	Slices are reachability over the dependency graph, and each statement keeps its inclusion reason.	[6], [7], [12]
analysis/pipeline.py: analyse_source, Program, FunctionAnalyses	Stages run in a fixed order over one program object; a stage that cannot complete records a reason instead of raising.	[4], [9]

4.5.3 Rules, Findings, Reporting and the Interface

Table 4.20: Rule, finding, reporting and interface modules.

Module and principal symbols	Invariant and output	Ref.
rules/model.py: RuleSet, SourceRule, SinkRule, SanitizerRule, FlowRule	A rule set is immutable data with a content fingerprint, displayed by the interface and recorded in every report.	[9], [10]
rules/rule_language.py: parse_rule_text, validate_declarations, load_builtin_rules	Rules are parsed by the project's LR parser and validated with line and column diagnostics before use.	[1], [2], [9]
rules/builtin/standard.ctrule	Five sources, eight sinks, five sanitizers, two validators and four flows, all visible and editable.	[9], [10]
findings/models.py: Finding, stable_finding_id, compare_findings	Identity derives from the rule, the sink operation and the statement text, not from the line number.	[9]
findings/explain.py: PathExplainer, build_findings, TraceStep	A finding carries an ordered path, branch conditions, certainty, limitations and remediation.	[6], [9]
reporting/json_report.py: build_report, write_json_report, dispositions	The report follows the documented schema, and a suppressed flow is reported as a disposition rather than silently dropped.	—
reporting/html_report.py: render_html, write_html_report	The HTML report is a single file, without script, in which all analysed text is escaped.	—
api/server.py: CodeTraceHandler, RunStore, create_server	Requests are validated before the engine runs; limits and error codes are documented; runs are held in memory only.	—
api/schemas.py: the schema catalogue	Schemas are data, so the documentation endpoint and the tests read the same description.	—
frontend/js/app.js: router, run lifecycle, theme	One run identifier is shared by every view, and stale text is flagged before a result is quoted.	—
frontend/js/state.js: the store	Only the store holds analysis results, so views cannot disagree about a run.	—
frontend/js/graph.js, js/lib/layout.js: renderer and layered layout	Deterministic positions for one graph; nodes are focusable and selectable.	—
frontend/js/views/: the nine views	A view renders state and API data; it never re-derives an analysis fact.	—

4.6 Worked Example: Command Injection

The example below is one of the bundled programs. A helper function builds a command string from a value read from standard input, and the result is passed to the shell. Comments and unrelated lines have been removed so that the path is easy to follow.

examples/vulnerable/command_injection.ctc

```
string build_command(string user_input) {
  return "ls " + user_input;
}

int main() {
  string name = input();
  string command = build_command(name);
  system(command);
  return 0;
}
```

The rule set declares input as a source of untrusted data in the command, SQL, path, HTML, numeric and format contexts, and it declares system with argument zero as a sink in the command context. The analysis reports one HIGH finding in the command-injection category with certainty approximate, because the value crosses a function boundary and the summary is context-insensitive. The eight steps below are the explanation that accompanies it.

Table 4.21: The evidence path reported for the worked example.

Step	Kind	Location	What the step records
1	source	main, 7:17	input introduces untrusted data under the source rule stdin.
2	declaration	main, 7:3	name is assigned a tainted value.
3	argument	main, 8:34	Argument zero of the call to build_command is tainted.
4	parameter	build_command, 2:22	user_input receives the value through the parameter list.
5	return	build_command, 3:3	The returned expression carries the same taint atom.
6	call	main, 8:20	The function summary propagates taint to the caller.
7	declaration	main, 8:3	command is assigned the tainted result.
8	sink	main, 9:10	system consumes, in argument zero, data that is not safe for the command context.

Finding FEF13A28F7E0E, rule command_execution, severity HIGH, certainty approximate. The run reports 43 tokens, 22 syntax nodes, 5 statements, 4 basic blocks, 3 definitions, 10 dependency nodes and 10 dependency edges.

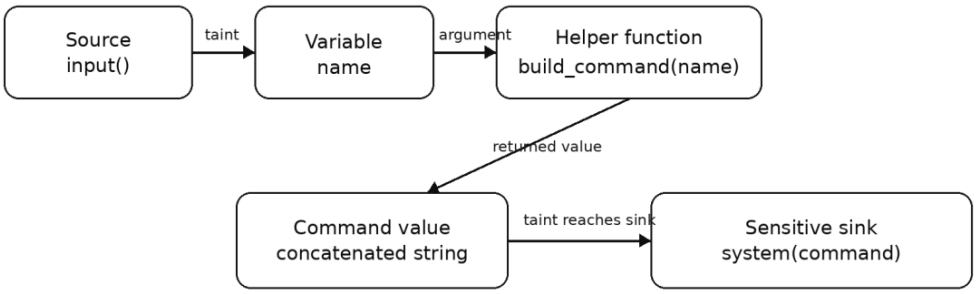


Figure 4.1: The reported flow from source to sink, drawn from the run whose evidence path is tabulated above.

The same run is shown in two forms. Figure 4.2 is the dependency graph the engine computed for the file, redrawn from the analysis output so that the data, call, return and taint edges can be told apart; Figure 5.8 is the interactive view of the same graph in the workbench, where a node can be selected to read the reaching definitions and live variables at the corresponding statement.

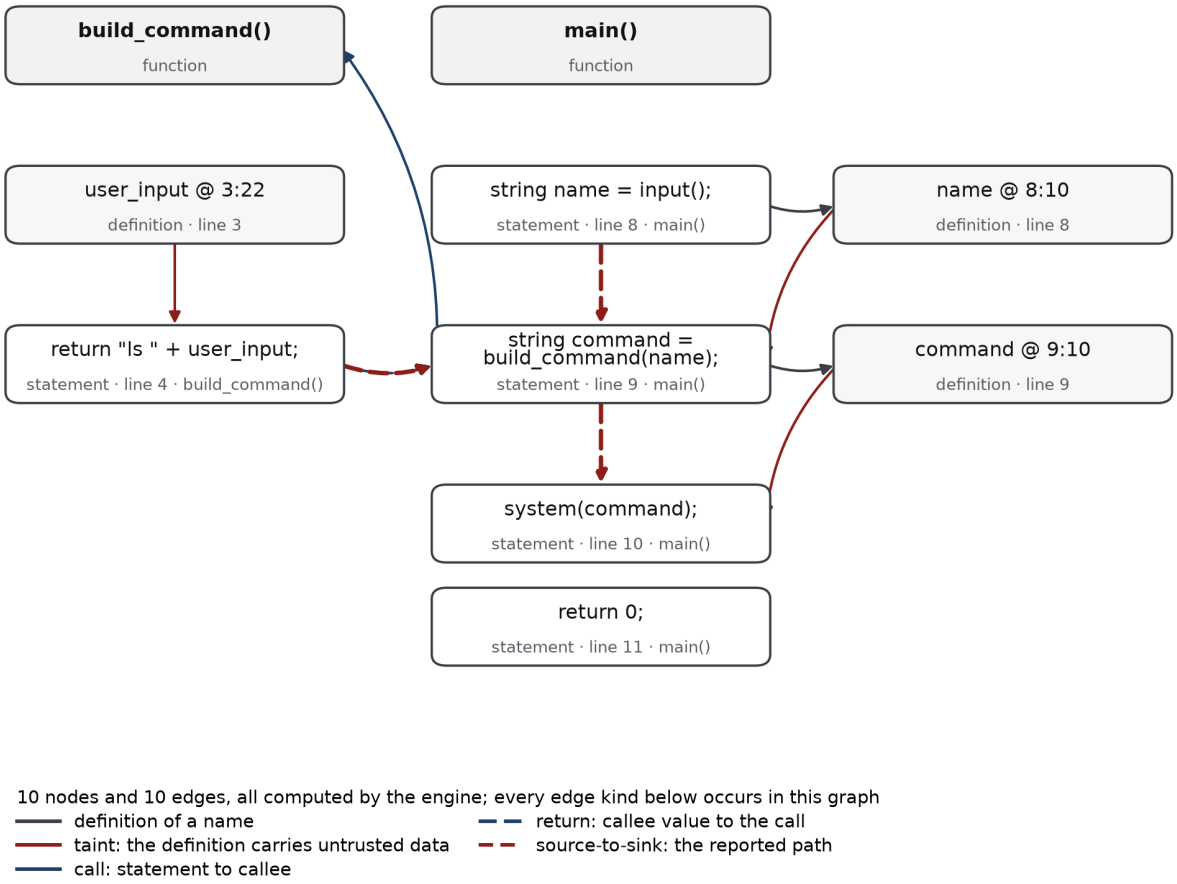


Figure 4.2: The typed dependency graph of the worked example, drawn from the graph the engine computed: ten nodes, ten edges, and the reported source-to-sink path in dashes.

4.7 Diagnostics and Partial Analysis

A refusal is as informative as a result, provided it is specific. The engine reports diagnostics with a code, a phase, a position and a message, and it marks a run partial when a diagnostic or an analysis limit affects what can be concluded. The following cases are drawn from the bundled examples.

Table 4.22: Representative diagnostics, their phases and their effect on the run.

Example	Codes	Effect
syntax_errors/missing_semicolon.ctc	SYN001	The grammar no longer matches at 3:3; two located errors, the run is marked partial, and no finding is claimed.
syntax_errors/unbalanced_braces.ctc	SYN001, SEM020	The block structure is incomplete; the semantic phase reports the consequence rather than inventing a scope.
syntax_errors/broken_expression.ctc	SYN001	The expression cannot be derived; recovery keeps the remaining statements parseable.
unsupported/switch_statement.ctc	LEX100, LEX102, SYN100	The construct is outside CT-C and is named, rather than silently ignored.
unsupported/preprocessor_directive.ctc	LEX101	Preprocessor semantics are outside the language model; the directive is reported.
unsupported/pointer_type.ctc	SYN001	Pointer types are not in the grammar, and no pointer analysis is claimed.
challenges/shadowed_variable.ctc	SEM018	Shadowing is legal but reported, because it changes the symbol a later statement refers to.
challenges/array_element_taint.ctc	DFA001	An element may be read before it is written; arrays are not tracked element by element.

The design principle behind these messages is that the tool should never be able to say less than it knows. A construct it cannot model is reported as unsupported rather than skipped; a recovered parse is described as partial; and a sink that was inspected and not reported appears in the disposition panel with the reason, which is the subject of Section 4.7.

4.8 Reproducing the Results

Every claim in this report can be reproduced from a clean checkout with Python 3.11 and, for the interface suite only, Node.js 18 or newer. The commands below are the ones used to produce the figures in Chapter 5 and the tables in Section 5.3.

Commands used for the results in this report.

```
python3 -m codetrace.api                  # start the workbench on port 8000
scripts/tests.sh                          # every suite
python3 -m unittest discover -s tests -t .  # engine and integration suites only
cd frontend && npm test                   # interface suite against a real engine
scripts/evaluate.sh                       # the labelled corpus, JSON evidence in var/
python3 scripts/audit.py                  # independent audit, 69 checks
python3 scripts/mutation_check.py         # 18 deliberate defects: does the suite notice?
```

The screenshots were captured from the running server by scripts/capture_screenshots.js, which drives the workbench through the same steps a reviewer would take: load an example, run the analysis, open each view, select a finding and take a slice. The script fails if the page logs an error or a request fails, so a capture cannot silently record a broken page.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Application Walkthrough

The screenshots in this section were captured from the running system on this checkout, at a viewport of 1440 by 1000 pixels and a device scale of two. Each caption states what the view is for; the accompanying paragraph states what can be concluded from it. The header visible in every capture reports the engine version, the rule-set fingerprint and the size of the parsed grammar, so a reader can tell which build produced the picture.

5.1.1 Overview of the Workbench

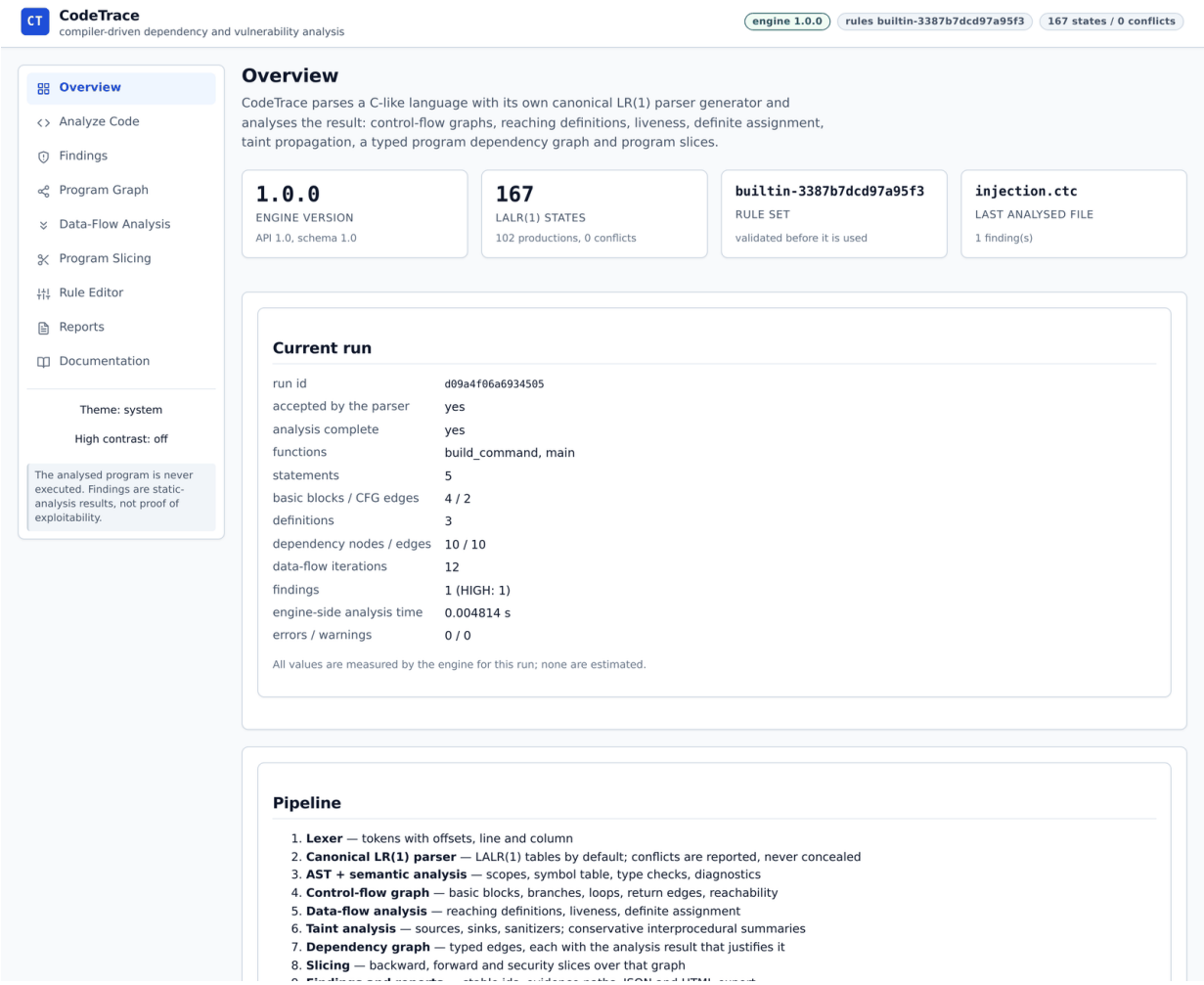


Figure 5.1: Overview of the Workbench.

The landing view identifies the build and reports the current run. The four cards give the engine version, the number of LALR states, the rule-set fingerprint and the file that was last analysed; the panel beneath them repeats the run summary that the API returned, including the token counts and the analysis time measured by the engine.

Nothing on this page is computed by the browser. Each value is a field of a response, which is why the summary can be compared directly with the JSON report.

5.1.2 Analysis Workbench

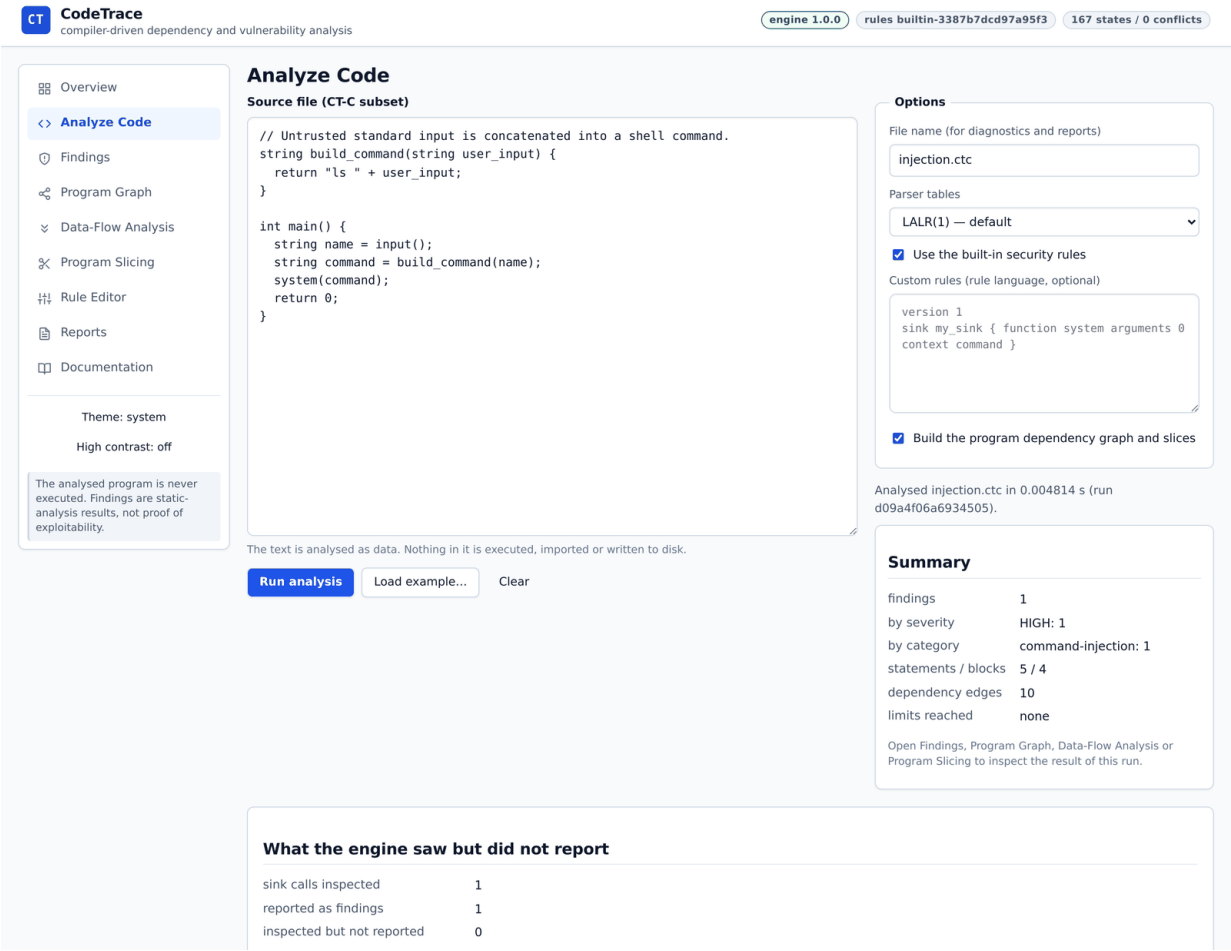


Figure 5.2: Analysis Workbench.

The editor holds the CT-C source and the panel beside it sets the display name, the parser mode, whether the built-in rules apply, an optional rule file and whether the dependency graph and slices should be built. The summary and the diagnostics for the run appear below the editor.

The parser mode is a real choice rather than a label: selecting the canonical mode rebuilds 656 states against the 167 of the default, and the analysis is otherwise unchanged.

5.1.3 Editor State After a Run

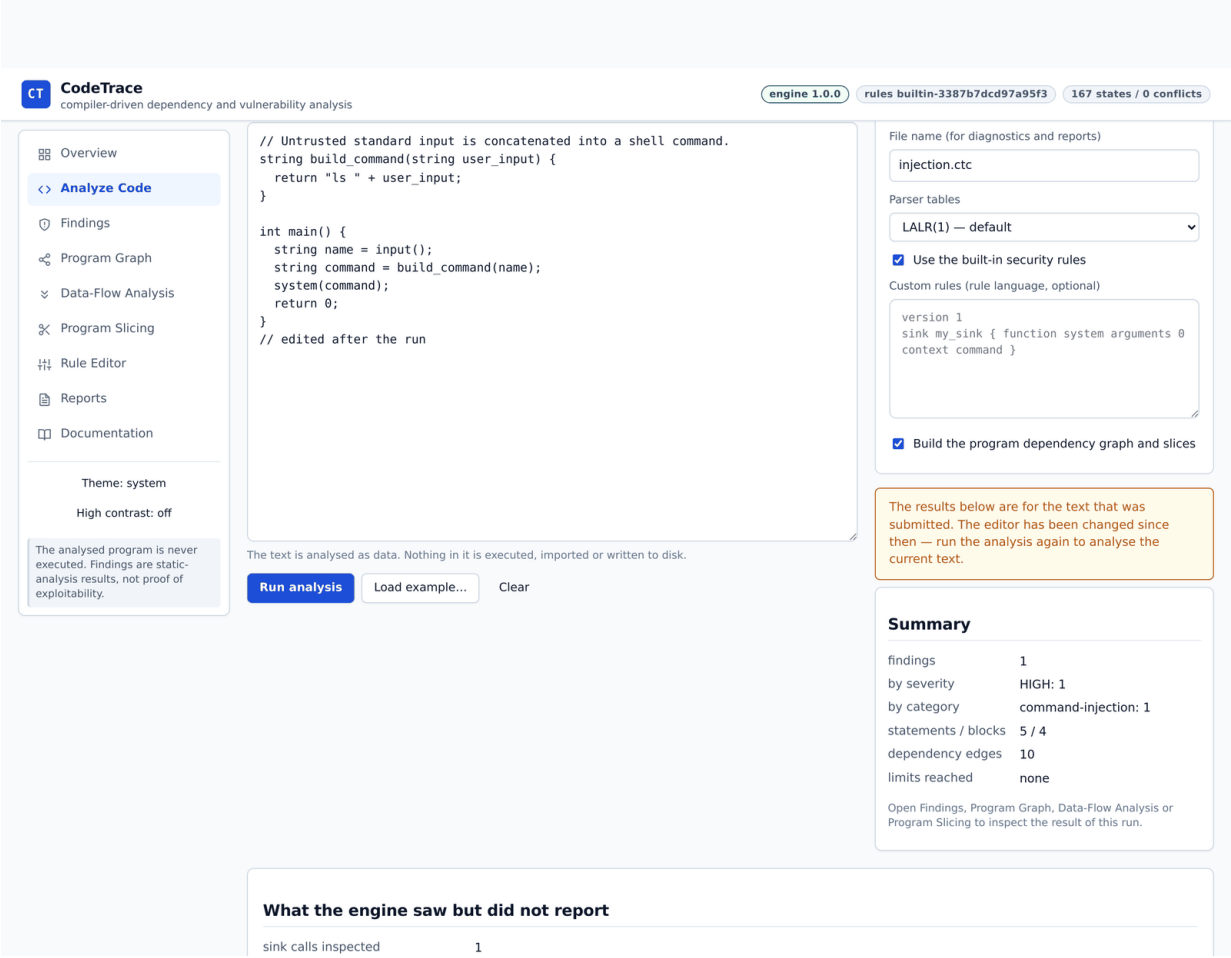


Figure 5.3: Editor State After a Run.

When the text is modified after a run, the workbench states that the results describe the text that was submitted and asks for the analysis to be repeated. The warning is contained within the options column and the previous results remain visible.

This is the defect corrected during Review 3. The warning had been rendered inline, so its text overflowed its column; the corrected rendering is the one shown here.

5.1.4 Located Syntax Diagnostics

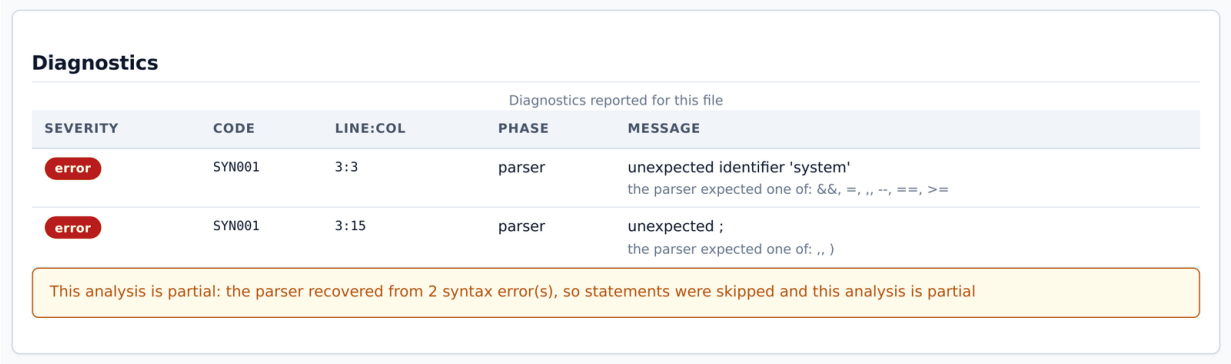


Figure 5.4: Located Syntax Diagnostics.

A file with a missing semicolon produces two located diagnostics rather than one. The first reports the unexpected identifier at 3:3 with the tokens that would have been accepted; the second reports the stray semicolon that follows.

Recovery is visible as a benefit: because the parser continued, both errors are reported in one pass, and the note beneath the table records that the analysis is partial.

5.1.5 A Refused Analysis

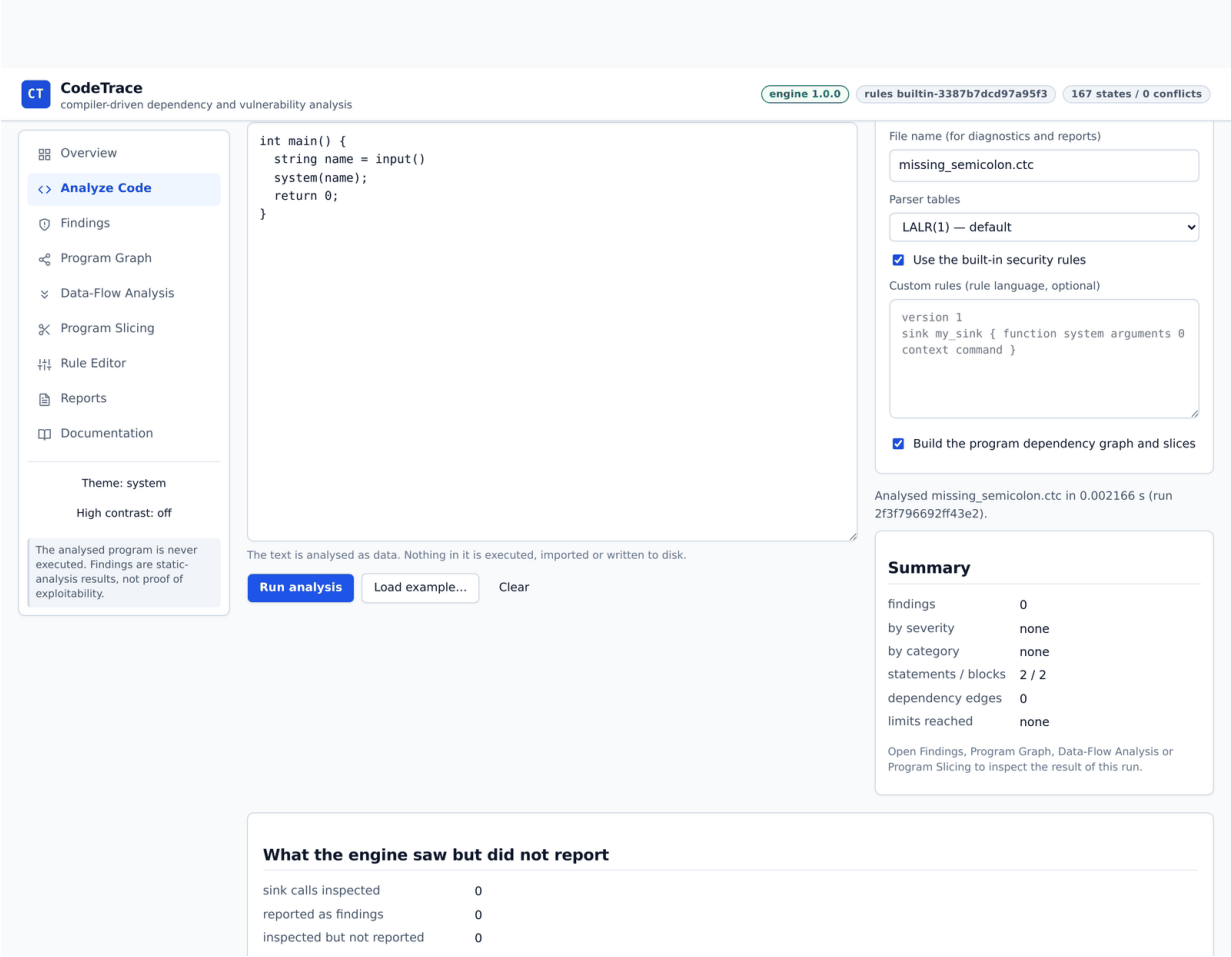


Figure 5.5: A Refused Analysis.

The same run in context: the summary reports zero findings, the status line records that the file was analysed with two recovered syntax errors, and the disposition panel states how many sink calls were inspected.

The combination matters. A reader who saw only the zero would draw a different conclusion from a reader who also sees that the parse was partial.

5.1.6 Findings with Evidence

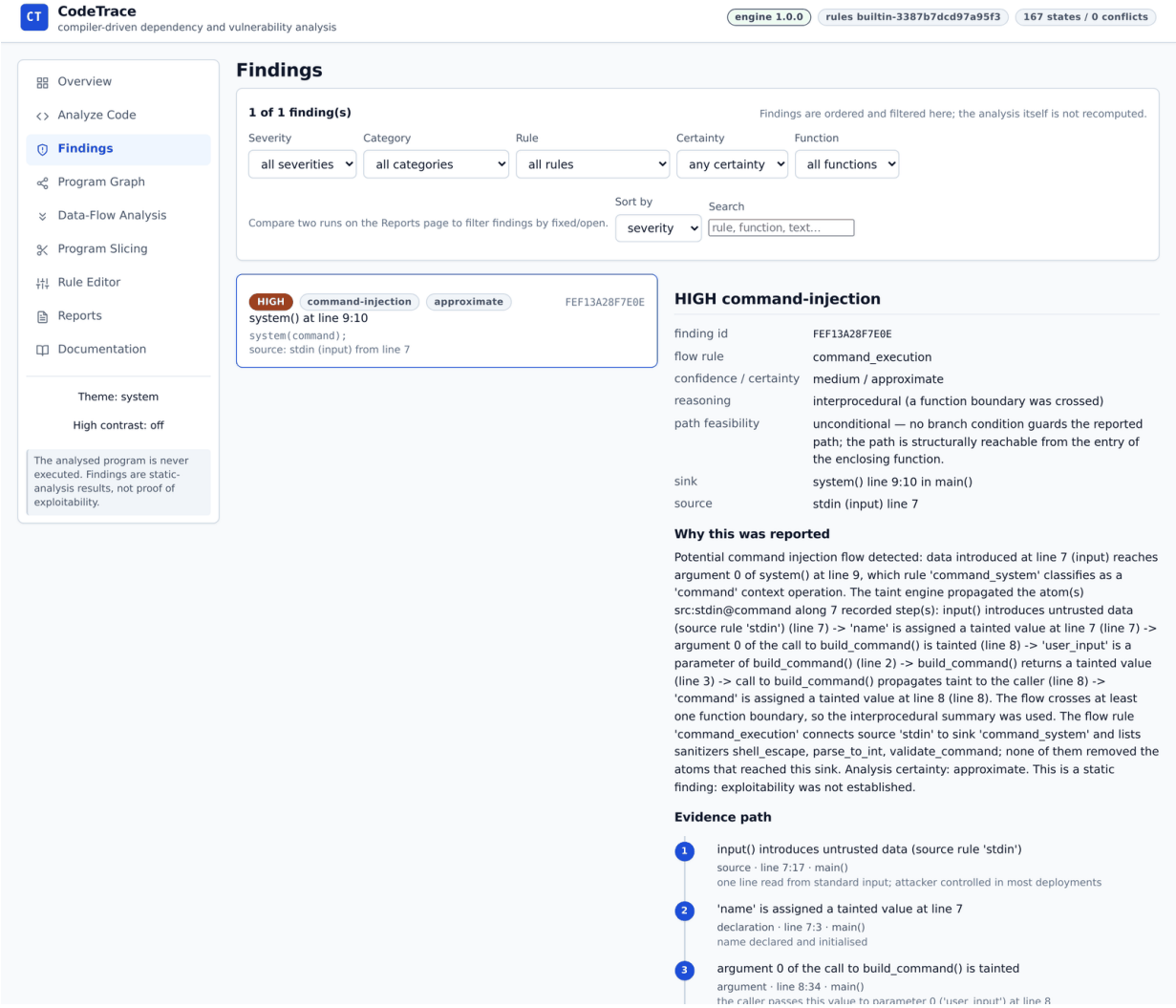


Figure 5.6: Findings with Evidence.

The list can be filtered by severity, category, rule, certainty and function, and sorted; the detail panel shows the identity, category, severity, certainty, sink, source, the reasoning and the beginning of the evidence path for the selected finding.

The reasoning paragraph is generated from the recorded facts, including the sanitizers that were considered and the reason the finding is approximate.

5.1.7 The Ordered Evidence Path



Figure 5.7: The Ordered Evidence Path.

Each step of the path is a statement with a kind, a position, the enclosing function and the analysis fact that admitted it: a source call, a declaration, an argument at a call site, a parameter, a return, a summary application and finally the sink.

The path is the explanation the report promised in Chapter 1. It can be read without the dependency graph, and it can be checked against the graph when more detail is wanted.

5.1.8 Program Dependency Graph

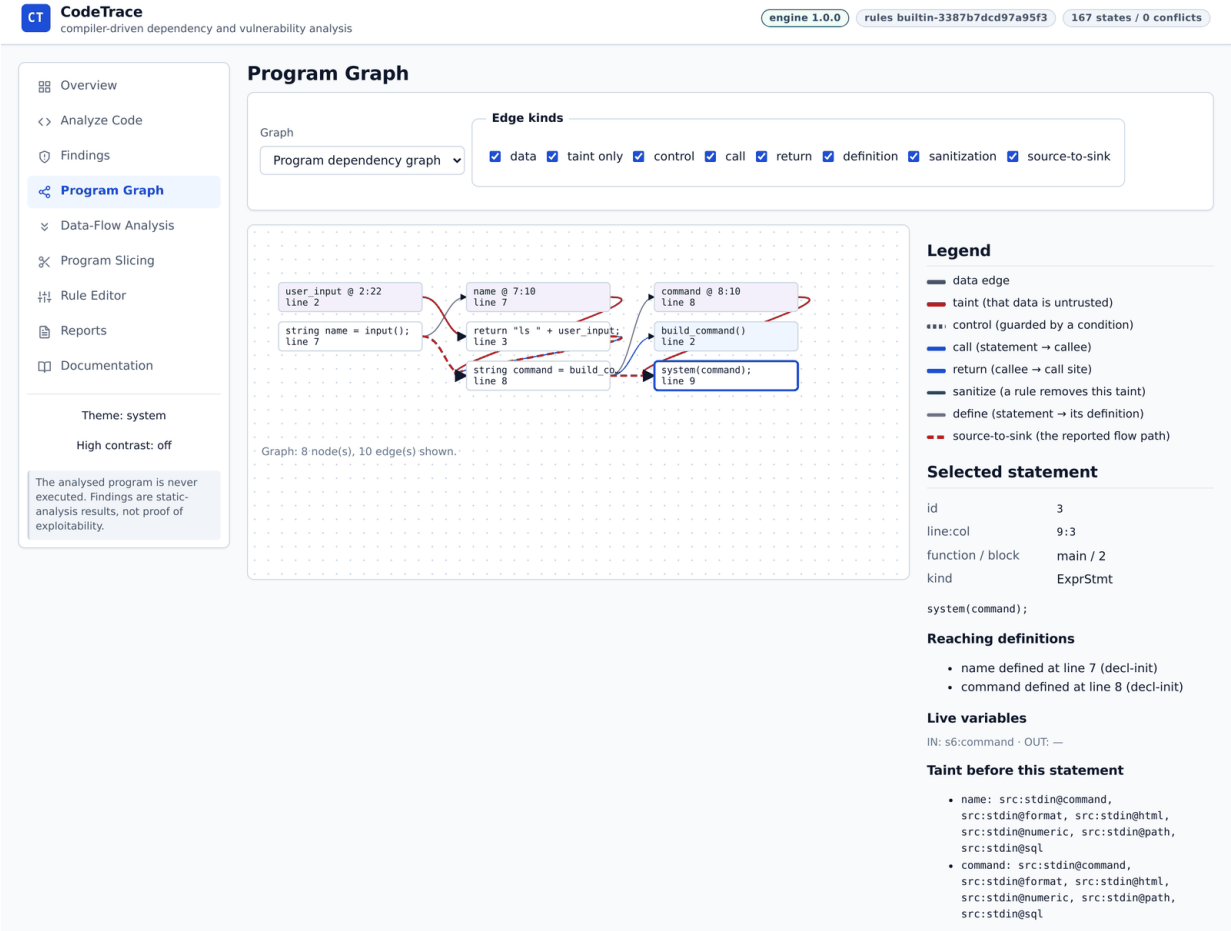


Figure 5.8: Program Dependency Graph.

The graph view draws the typed dependency graph with a legend that distinguishes data, taint, control, call, return, definition and sanitisation edges. Edge kinds can be switched off, and the count of drawn nodes and edges is reported below the canvas.

The dashed red edges are the reported source-to-sink path; the solid edges are the dependencies that support it. This separation is what allows a reader to see why the flow was reported without losing the context in which it occurs.

5.1.9 Data-Flow Analysis

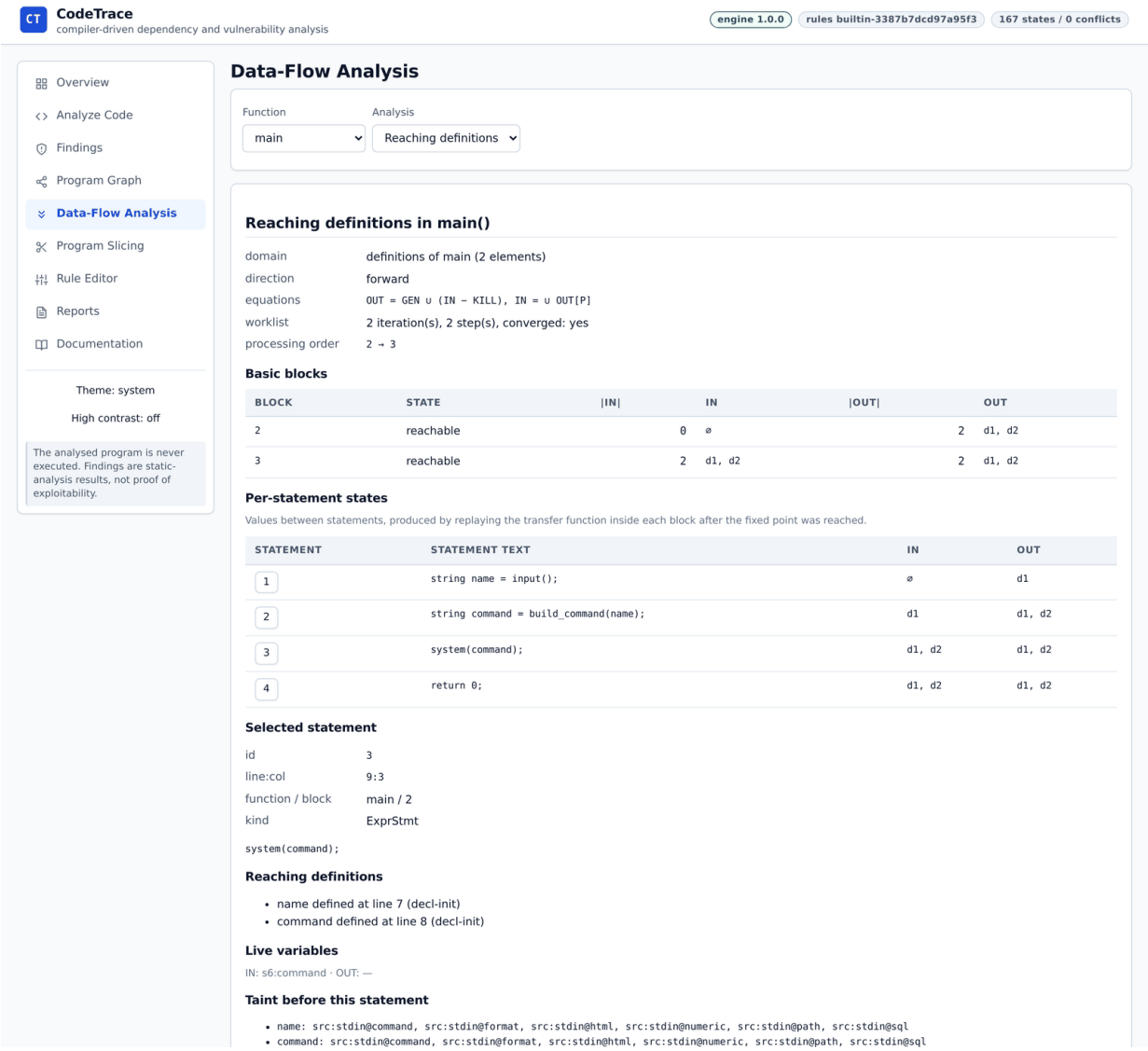


Figure 5.9: Data-Flow Analysis.

For the selected function the inspector states the domain, the direction, the equations, the number of worklist iterations and the processing order, then the IN and OUT set of each basic block and the state between statements. Selecting a statement shows its definitions, its live variables and the taint atoms that hold before it.

The numbers are the fixed point of the solver, which is why the interface can show the iteration count: the analysis is a computation with a trace, not a table of assertions.

5.1.10 Program Slicing

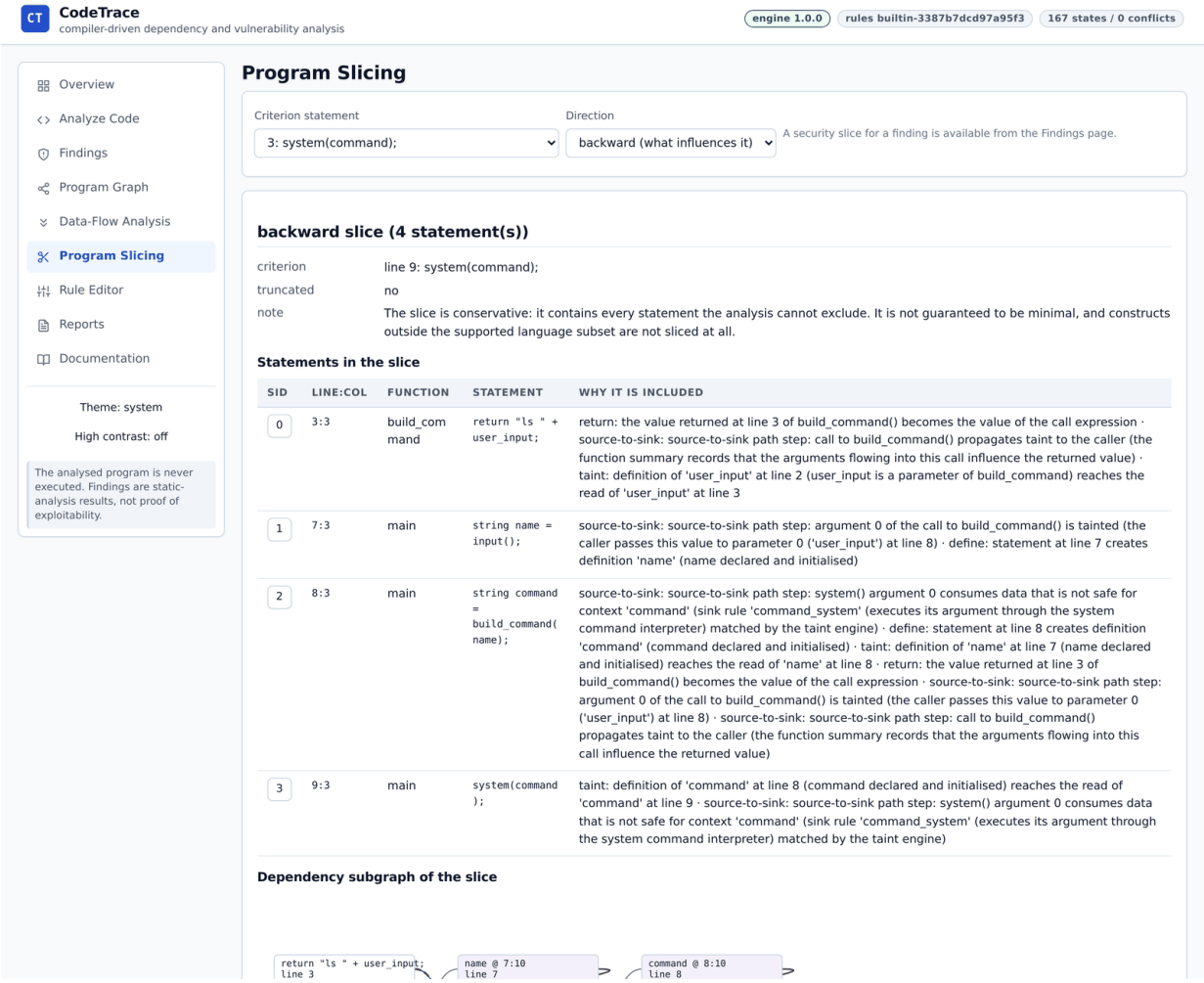


Figure 5.10: Program Slicing.

The slice view takes a criterion statement and a direction and reports the statements that remain, together with the reason each one was included and the conservative note that qualifies the result.

Including the reasons is what makes a slice reviewable. A statement appears because a definition reaches it, a call summary connects it, or a taint path passes through it, and the table says which.

5.1.11 The Induced Subgraph

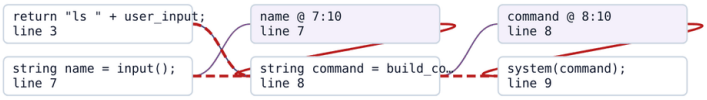


Figure 5.11: The Induced Subgraph.

Below the table the view draws the dependency subgraph induced by the slice, restricted to the statements that survived.

A slice that has been reduced to four statements and six edges can be read in a few seconds, which is the practical purpose of the technique.

5.1.12 Rule Editor and Validation

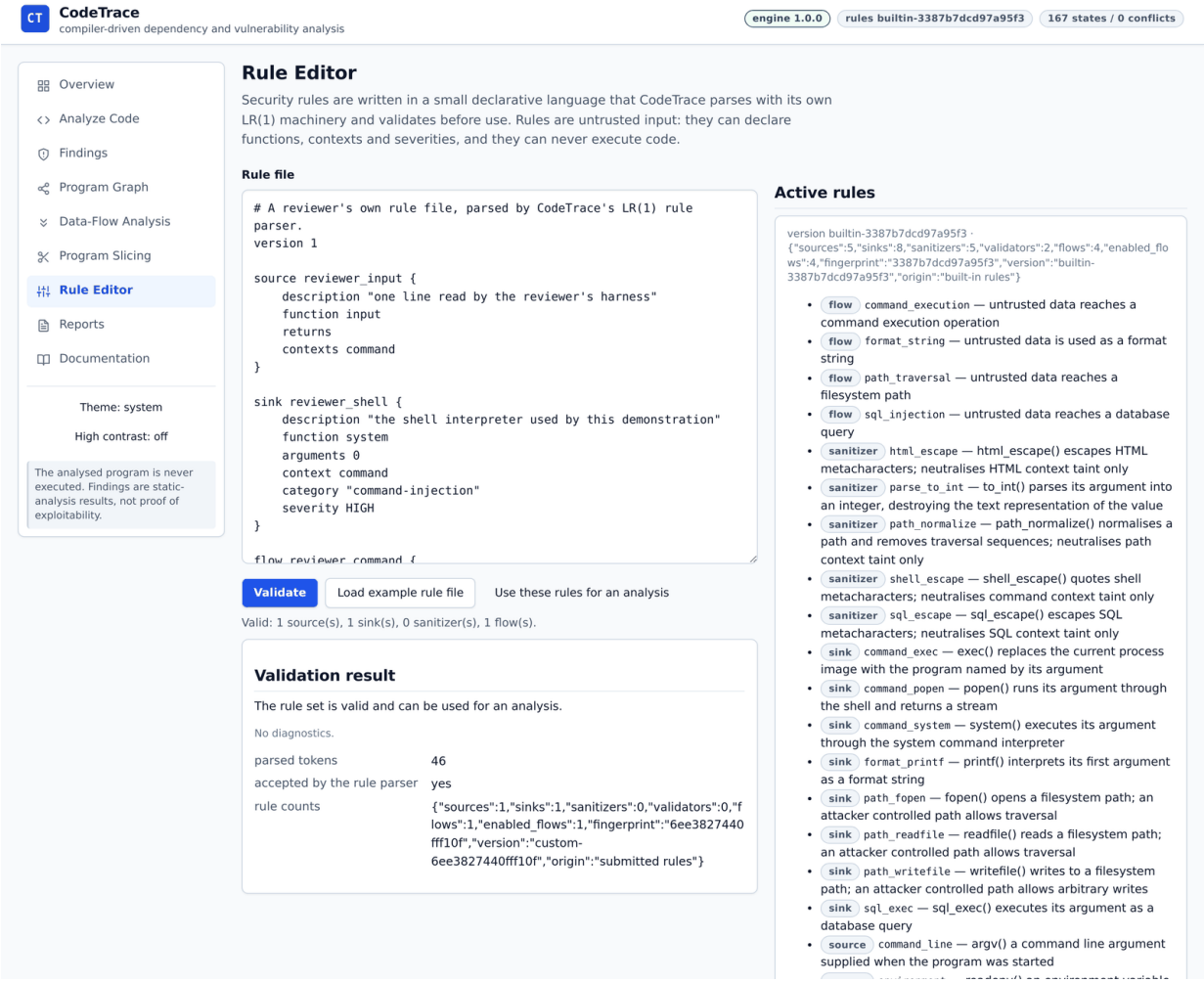


Figure 5.12: Rule Editor and Validation.

The editor accepts a rule file and reports whether it is valid, with the counts of sources, sinks, sanitizers and flows that it declares. The catalogue of rules in force is shown beside the editor.

The rules in the catalogue are the same declarations that appear in the built-in rule file; the language is not a second, hidden configuration mechanism.

5.1.13 Validation Result and Diagnostics



Figure 5.13: Validation Result and Diagnostics.

When a rule file is rejected, the diagnostics are listed with a code, a position and the token that would have been accepted, in the same form as a syntax error in CT-C.

This is the practical benefit of parsing the rule language with the same machinery: a mistake in a rule file is reported as precisely as a mistake in the analysed program.

5.1.14 Report Export and Run Comparison

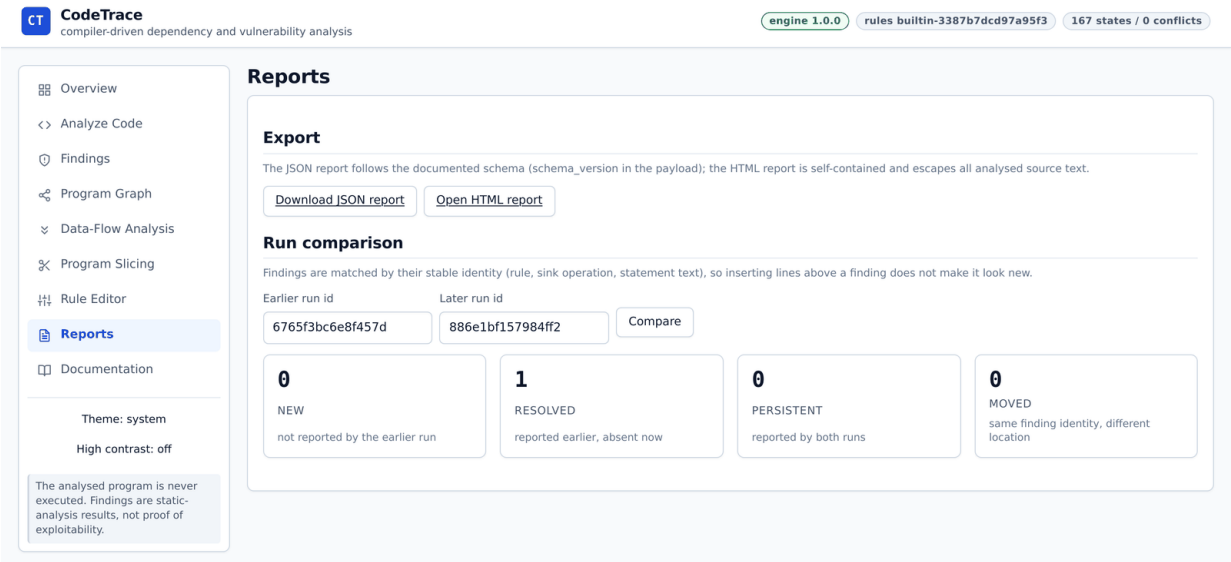


Figure 5.14: Report Export and Run Comparison.

A run can be exported as JSON, which follows the documented schema, or as a single self-contained HTML document. Two runs can be compared, and findings are classified as new, resolved, persistent or moved.

The comparison in the figure shows one resolved finding after the call to the shell was replaced by a constant. Because identity depends on the statement rather than on the line, the result is a statement about the change that was made.

5.1.15 Documentation View

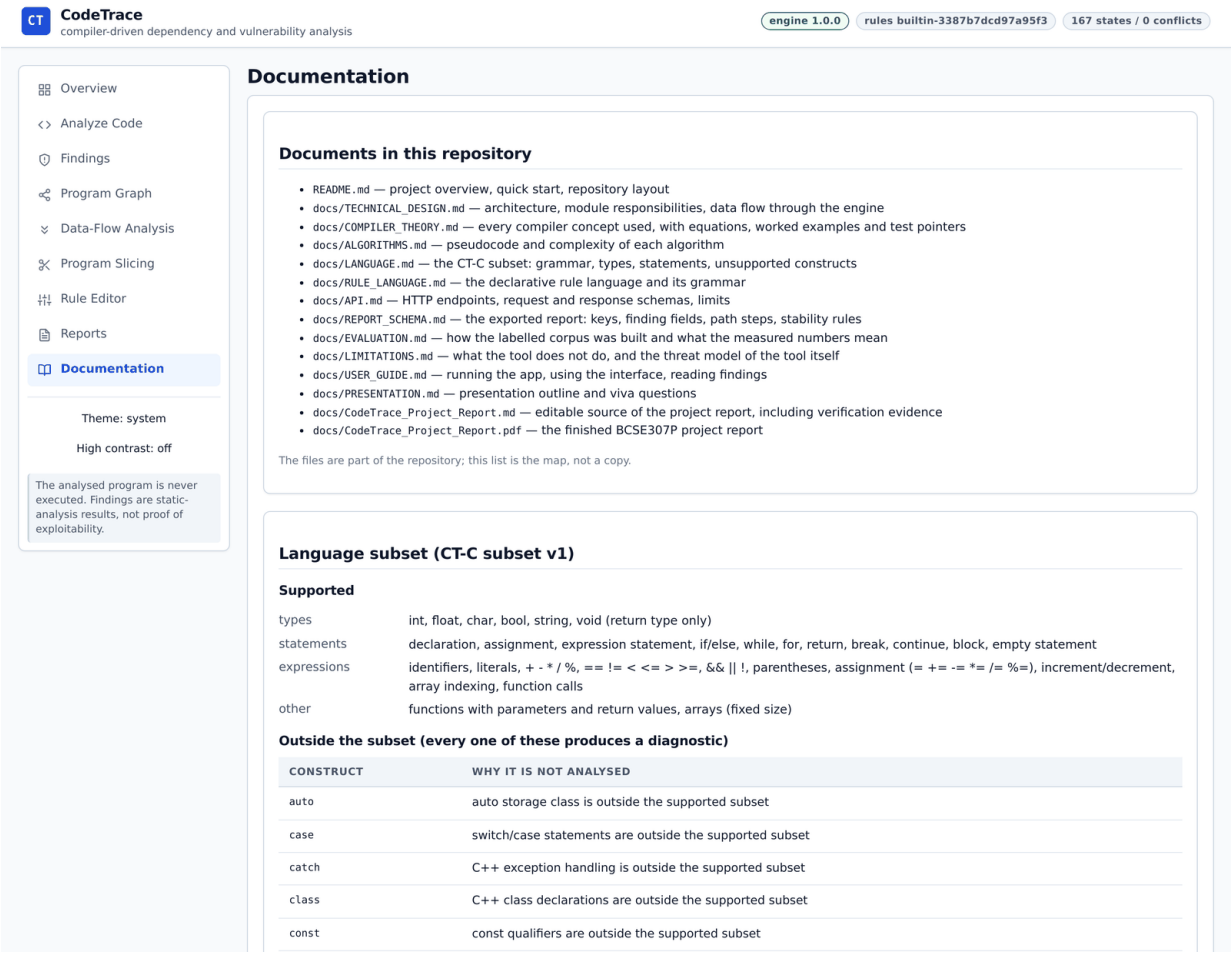


Figure 5.15: Documentation View.

The documentation view lists the documents in the repository and states the supported language: the types, statements, expressions and external functions, together with the constructs that are rejected.

The language description is served by the engine, so the page cannot describe a language that differs from the one the parser implements.

5.1.16 Dark Theme

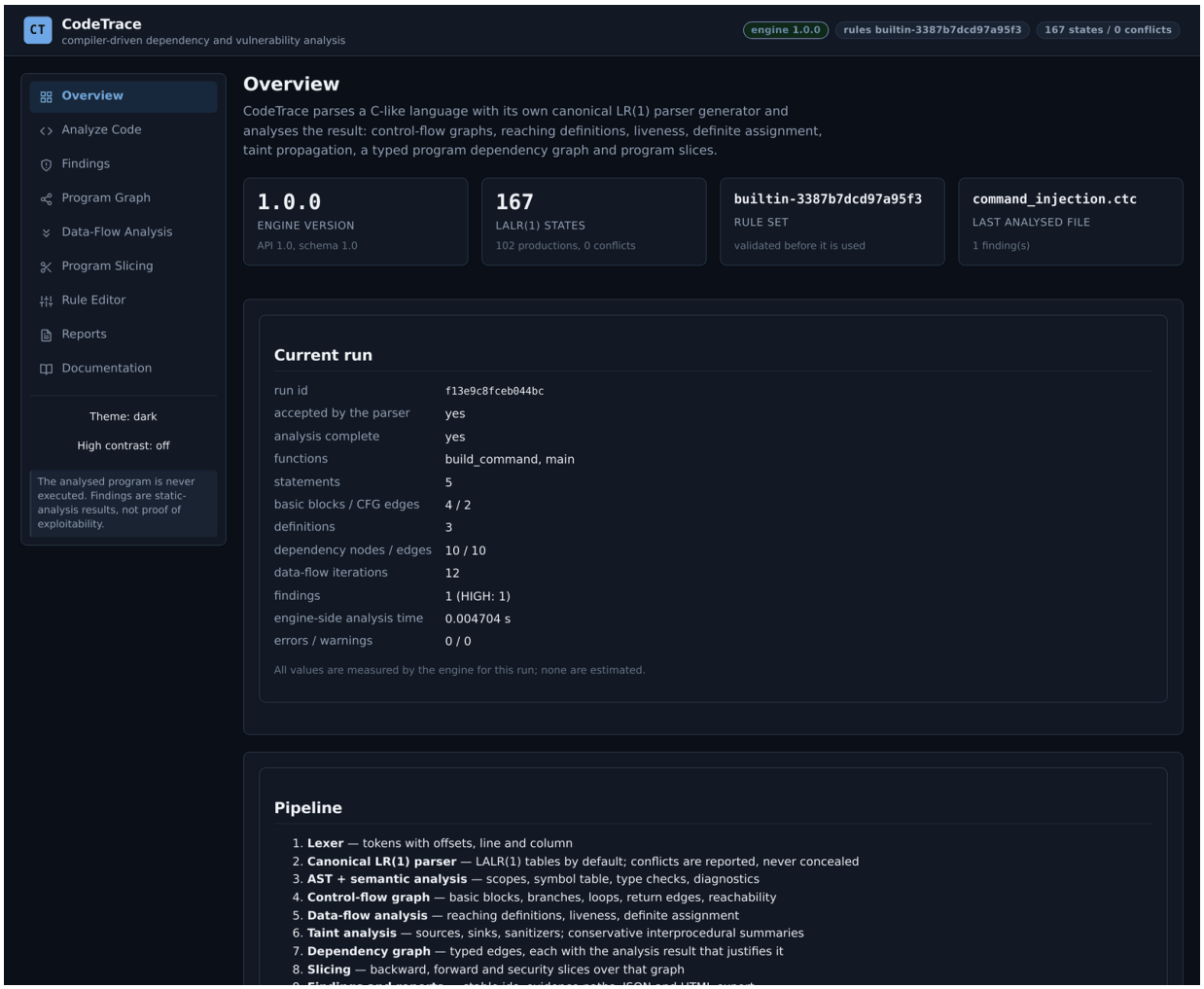


Figure 5.16: Dark Theme.

The workbench carries a dark theme and a high-contrast mode; the theme is a property of the document root, so every view inherits it.

The run summary in this capture is the same as the one in Figure 5.1, which demonstrates that presentation is separate from analysis.

5.2 Features Implemented

The table below maps each feature visible in the walkthrough to the module that implements it, the figure that shows it and the test suite that exercises it. Every row corresponds to a capability that was added for the review rather than to a planned one.

Table 5.1: Features implemented, with the module that provides each and the evidence for it.

Feature	Implemented in	Evidence: figure and suite
Run summary and pipeline overview	frontend/js/views/overview.js	Figure 5.1; tests/test_api.py
Analysis workbench with parser and rule options	frontend/js/views/analyze.js	Figure 5.2; tests/test_api.py, frontend/tests/views.test.js
Editor-state guard after a run	frontend/js/app.js, css/app.css	Figure 5.3; frontend/tests/regressions.test.js
Located syntax diagnostics and partial status	codetrace/frontend/lexer.py, clr_parser.py	Figure 5.4; tests/test_lexer.py, tests/test_parser.py
Findings with certainty, limitations and remediation	codetrace/findings/explain.py	Figure 5.6; tests/test_workflow.py, tests/test_taint.py
Ordered evidence path	codetrace/findings/explain.py	Figure 5.7; tests/test_workflow.py
Typed dependency graph with edge filters	codetrace/analysis/dependencies.py, frontend/js/graph.js	Figure 5.8; tests/test_slicing.py, frontend/tests/views.test.js
Data-flow inspector with per-statement states	codetrace/analysis/dataflow.py, frontend/js/views/dataflow.js	Figure 5.9; tests/test_dataflow.py
Slicing with inclusion reasons and subgraph	codetrace/analysis/slicing.py, frontend/js/views/slicing.js	Figures 5.10 and 5.11; tests/test_slicing.py
Rule validation with located diagnostics	codetrace/rules/rule_language.py, frontend/js/views/rules.js	Figures 5.12 and 5.13; tests/test_rules.py
JSON and HTML export, run comparison	codetrace/reporting, frontend/js/views/reports.js	Figure 5.14; tests/test_reporting.py
Language and diagnostics documentation from the engine	codetrace/api/server.py, frontend/js/views/docs.js	Figure 5.15; tests/test_api.py
Dark and high-contrast themes	frontend/css/app.css	Figure 5.16; frontend/tests/views.test.js

5.3 Verification and Testing

Four instruments are used, and they are deliberately different in kind. The test suites check the behaviour of the code; the audit checks the claims the documents make about the code; the mutation check asks whether the suites would notice a plausible mistake; and the corpus measures accuracy against expectations that were written before the analysis was run.

Table 5.2: Verification instruments and their results on this checkout.

Instrument	Command	Result
Engine and integration suites	python3 -m unittest discover -s tests -t .	350 tests, all passing
Interface suite	npm test, in frontend/	53 tests, all passing
Independent audit	python3 scripts/audit.py	69 checks, all passing
Mutation check	python3 scripts/mutation_check.py	18 of 18 defects detected
Labelled corpus	python3 evaluation/run_evaluation.py	33 files, precision 1.0, recall 1.0

The audit deserves a note of explanation. It does not import the test suite; it re-reads the documents, the example expectations and the table of modules, and it fails when a quoted number no longer matches the tree. During this review it caught a stale line count in the technical design document, which was corrected rather than exempted, and the check now passes. The mutation check works in the opposite direction: it edits the engine in eighteen small and plausible ways, one at a time, and requires the suite to fail for each of them. A surviving mutation would be recorded as a hole in the tests.

5.4 Evaluation Results

The corpus contains 33 CT-C programs, each carrying an expectation written by hand before the analysis was run. A case either declares the findings it should produce, or declares that it should produce none, or declares the diagnostics that should cause the analysis to be refused. Matching is by rule, category, severity, sink statement and source rule; the line number is recorded but is not part of the match, so a case does not pass merely because a line moved.

Table 5.3: Composition of the labelled corpus and the outcome for each group.

Group	Files	Expectation	Outcome
Vulnerable programs	17	20 findings in total	all reported, no spurious finding
Custom-rule program	1	1 finding under a submitted rule file	reported under the submitted rules
Safe programs	5	no finding	silent
Sanitized programs	4	no finding	silent, with the sanitizer recorded
Syntax-error programs	3	diagnostics only	refused, partial status
Unsupported constructs	3	diagnostics only	refused, construct named
Total	33	20 findings	precision 1.0, recall 1.0, F1 1.0

The harness measured about a second of analysis time over the whole corpus, and it prints the exact figure, which depends on the machine.

A precision of 1.0 and a recall of 1.0 on this corpus should be read for what they are: a statement that the engine and the expectations agree on 33 programs written for the project, not a statement about the accuracy of the analyser on unknown code. The corpus is small, it was written by the same team that wrote the engine, and several of its cases were added in response to a defect the team had found. Section 5.6 treats this as the principal threat to validity, and the planned work in Section 5.8 includes an independently authored corpus.

5.5 Discussion

Three observations follow from the results. The first concerns the value of exposing intermediate artefacts. During development, most of the defects that were found were found by reading a data-flow table or a dependency edge rather than by reading a failing assertion: the interface made an incorrect fact visible before a test was written for it. That is an argument for treating the artefacts as part of the product rather than as debugging aids.

The second concerns the limits of a corpus written by the authors. Agreement between an engine and its own expectations measures consistency, and consistency can be achieved by writing expectations that match the implementation. The mutation check is the counterweight: it asks whether the tests are sensitive to the engine, and it found two tests that were not, which were strengthened. A tool that only reports its corpus numbers would overstate its evidence.

The third concerns the cost of honesty in the interface. Reporting partial runs, unsupported constructs and inspected-but-unreported sinks makes the interface denser and makes the results

harder to summarise. It also prevents the most damaging misreading of a static analyser, in which an empty result is taken as a proof of safety. The design decision recorded in Table 3.7 is therefore deliberate, and the screenshots in Section 5.1 show what it costs in practice: a reader must look at the status line as well as at the findings.

5.6 Limitations and Threats to Validity

The limitations of the analysis are documented per finding and summarised here. They fall into three groups: the language, the analysis and the evidence.

Table 5.4: Limitations, their effect on a result and where they are disclosed.

Limitation	Effect on a result	Disclosed in
Small language subset	Programs using pointers, aliasing, the preprocessor or several translation units cannot be analysed as written.	Diagnostics, Section 4.7; docs/LANGUAGE.md
Conservative taint	Some real flows are not reported, because a path that the analysis cannot justify is not claimed.	Limitations list per finding
Context-insensitive summaries	A flow crossing a function boundary may describe a path that no single execution realises; certainty is marked approximate.	The finding, and Section 3.4.6
Arrays as whole objects	A write to one element taints the whole array, and a read of another element may be reported conservatively.	The finding, and docs/LIMITATIONS.md
No implicit flows	A value that influences control flow without being assigned is not tracked, which can hide a flow.	docs/LIMITATIONS.md
Validator assumptions	A validator is trusted to mean what its declaration says; the body of the validator is not analysed.	Dispositions panel; report summary
Authors' corpus	Precision and recall describe 33 programs written by the project team, not unknown code.	Section 5.4 and docs/EVALUATION.md
Single translation unit	Inter-file behaviour is outside the model.	docs/LANGUAGE.md

The threat to validity that matters most for the evaluation is the provenance of the corpus, and the second is the absence of a comparison with an established analyser on the same inputs. Both are addressed in the planned work rather than argued away. The internal consistency of the numbers is, however, protected: the audit checks the figures quoted in the documents against the tree, so a number cannot drift while the implementation changes.

5.7 Current Progress and Challenges

All modules described in Chapter 3 are implemented and integrated. The pipeline runs end to end from source text to a report, the nine views present every artefact it produces, and the verification instruments pass on this checkout. The remaining work is a matter of breadth and of independent evidence rather than of missing components.

Two challenges shaped the implementation more than any other. The first was preserving source locations and symbol identities across every stage, so that a statement identifier in a data-flow table, a node in the dependency graph and a step in an evidence path all refer to the same text. The second was defining where the analysis stops. Deciding that a pointer declaration is rejected, that an unknown external function keeps its taint, and that a validator suppresses a flow whose body was not examined required decisions that are now documented, tested and visible in the results.

5.8 Conclusion and Future Enhancements

The project set out to show that compiler construction and security analysis can be presented as one inspectable pipeline rather than as two disconnected subjects. The result is a system in which a hand-written lexer, a generated LR parser, a scoped semantic analyser, three data-flow solvers, a declarative taint analysis, a typed dependency graph and a slicer operate on the same representation of the same file, and in which every conclusion retains the evidence that produced it. The work is bounded by a small language and a conservative analysis, and the report states those bounds wherever they affect a result.

Future work should proceed in the order of the threats to validity. An independently authored corpus and a comparison against an established analyser would give the evaluation the external standing it currently lacks. Context-sensitive function summaries and stronger reasoning about branch conditions would improve precision where the analysis is now approximate. Finer treatment of arrays and a first step towards alias analysis would widen the language subset usefully without abandoning the property that every construct is either analysed or reported. Beyond the analysis, additional report formats and an export of the dependency graph for external tools would make the artefacts easier to use outside the workbench.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Boston, MA, USA: Pearson/Addison-Wesley, 2006.

[2] D. E. Knuth, “On the translation of languages from left to right,” Information and Control, vol. 8, no. 6, pp. 607–639, 1965.

[3] F. L. DeRemer, “Simple LR(k) grammars,” Communications of the ACM, vol. 14, no. 7, pp. 453–460, 1971.

[4] G. A. Kildall, “A unified approach to global program optimization,” in Proceedings of the ACM Symposium on Principles of Programming Languages (POPL), 1973, pp. 194–206.

[5] F. Nielson, H. R. Nielson, and C. Hankin, Principles of Program Analysis. Berlin, Germany: Springer, 1999.

[6] M. Weiser, “Program slicing,” IEEE Transactions on Software Engineering, vol. SE-10, no. 4, pp. 352–357, 1984.

[7] J. Ferrante, K. J. Ottenstein, and J. D. Warren, “The program dependence graph and its use in optimization,” ACM Transactions on Programming Languages and Systems, vol. 9, no. 3, pp. 319–349, 1987.

[8] T. Reps, S. Horwitz, and M. Sagiv, “Precise interprocedural dataflow analysis via graph reachability,” in Proceedings of the ACM Symposium on Principles of Programming Languages (POPL), 1995, pp. 49–61.

[9] B. Chess and G. McGraw, “Static analysis for security,” IEEE Security & Privacy, vol. 2, no. 6, pp. 76–79, 2004.

[10] MITRE, “Common Weakness Enumeration (CWE),” including CWE-22, CWE-78, CWE-89 and CWE-134. [Online]. Available: https://cwe.mitre.org/

[11] F. DeRemer and T. Pennello, “Efficient computation of LALR(1) look-ahead sets,” ACM Transactions on Programming Languages and Systems, vol. 4, no. 4, pp. 615–649, 1982.

[12] S. Horwitz, T. Reps, and D. Binkley, “Interprocedural slicing using dependence graphs,” ACM Transactions on Programming Languages and Systems, vol. 12, no. 1, pp. 26–60, 1990.

[13] V. B. Livshits and M. S. Lam, “Finding security vulnerabilities in Java applications with static analysis,” in Proceedings of the 14th USENIX Security Symposium, 2005, pp. 271–286.

[14] OWASP Foundation, “OWASP Top 10:2021, A03:2021 Injection.” [Online]. Available: https://owasp.org/Top10/